Embedded Artificial Intelligence in Battery Management Systems: Pruning and Quantization for Efficient State-of-Charge and State-of-Health Estimation

Anonymous

Anonymous,

Abstract

Online battery management requires accurate state-of-charge (SOC) and state-of-health (SOH) estimates within strict memory and timing budgets. To address these constraints, this work investigates how stateful long short-term memory (LSTM) networks with multi-layer perceptron (MLP) heads can be optimised for embedded artificial intelligence (AI), using structured pruning and recurrent-weight quantization to reduce memory and computational costs while retaining estimation accuracy on an STM32H753ZI microcontroller. Using the open *Lithium Iron Phosphate (LFP) Cycle Ageing Dataset*, full-precision models provide the accuracy reference against which the compressed variants are evaluated. Long-horizon replay yields mean absolute error (MAE) values of 2.3–2.8 percentage points for SOC and 0.9–1.5 for SOH, showing that compression largely preserves accuracy. Hardware-in-the-loop benchmarks show that the evaluated 30 % pruning configuration reduces flash by 40.9 % and 45.5 % and inference time by 42.9 % and 44.0 % for the SOC and SOH models, respectively. The analytical complexity model links these savings to pruning fraction and network dimensions. Quantization further reduces flash by 50.2 % and 58.8 %, but raises runtime in the mixed-precision implementation. Together, the study provides a reproducible reference for resource-efficient embedded battery-state estimation.

Keywords: Embedded artificial intelligence, Battery state estimation, Long short-term memory network, Structured pruning, Weight quantization, Microcontroller deployment

Nomenclature

Acronyms

BMS	Battery management system
SOC	State of charge
SOH	State of health
DoE	Design of Experiments
LFP	Lithium iron phosphate
LSTM	Long short-term memory network
MLP	Multi-layer perceptron
STM32	32-bit ARM Cortex-M microcontroller family
MAE	Mean absolute error
RMSE	Root-mean-square error
MAC	Multiply–accumulate operation
INT8	Signed 8-bit integer representation (quantized weights)
FP32	32-bit floating-point representation

Symbols

t	Time
$U(t)$	Terminal voltage at time t
$I(t)$	Current at time t (positive for discharge)
$T(t)$	Cell temperature at time t
EFC(t)	Equivalent full cycles accumulated up to t
$Q_c(t)$	Cumulative charge throughput up to t
C_{nom}	Nominal discharge capacity of the cell
C_{app}	Apparent capacity inferred from the latest check-up
DoD	Depth of discharge

$\mathrm{SOC}(t)$	State of charge at time t
SOH_k	State of health at reference cycle k
$\mathbf{x}_t$	Input feature vector at time t
$\mathbf{h}_t$	LSTM hidden state at time t
$\mathbf{c}_t$	LSTM cell state at time t
$\mathbf{z}_t$	Concatenation of input and previous hidden state at time t
$\mathbf{i}_t, \mathbf{f}_t, \mathbf{o}_t, \mathbf{g}_t$	Input, forget, output, and candidate gates at time t
$\sigma(\cdot)$	Logistic sigmoid activation function
$\tanh(\cdot)$	Hyperbolic tangent activation function
$\odot$	Element-wise (Hadamard) product
W	Weight matrix (LSTM or MLP)
b	Bias vector
s	Per-row quantization scale (FP32)
$W^{\mathrm{fp32}}, W^{\mathrm{int8}}$	Original FP32 weight matrix and its stored INT8 representation
$\hat{W}$	Floating-point reconstruction of an INT8 weight matrix
ε_q	Positive lower bound for the quantization scale
$\mathcal{R}$	Index set of retained LSTM hidden units
e_n	Signed prediction error of sample n in percentage points
$Q_p(\cdot)$	Empirical quantile operator at probability p
$W_{\mathrm{ih}}^{\mathrm{int8}}$	Quantized input-to-gate weight matrix
$W_{\mathrm{hh}}^{\mathrm{int8}}$	Quantized hidden-to-gate weight matrix
$s_{\mathrm{ih}}, s_{\mathrm{hh}}$	Per-row quantization scale vectors for $W_{\mathrm{ih}}^{\mathrm{int8}}$ and $W_{\mathrm{hh}}^{\mathrm{int8}}$
b_{fp32}	Floating-point bias of a given gate
$\mathbf{a}_t$	Stacked LSTM gate pre-activation vector at time t
D, H, M	Input dimension, LSTM hidden size, and MLP width
N_{MAC}	Analytical multiply–accumulate count per inference

1. Introduction

Accurate online estimation of the state of charge (SOC) and the state of health (SOH) remains a central challenge for battery-powered systems [1, 2, 3, 4]. Recent reviews show sustained progress in model-based and data-driven estimators, but they also emphasise that accuracy depends strongly on cell chemistry, operating conditions, reference construction, and evaluation protocol [5, 1]. For an embedded battery management system, predictive accuracy alone is therefore insufficient: the estimator must also fit the available flash and RAM, meet the update deadline, and preserve its state consistently over long input streams.

The study is motivated by modular stationary storage systems in remote microgrids. Within the MG Farm programme [6, 7, 8, 9], partners from France, Algeria, Morocco, and Germany develop smart-grid demonstrators for North African sites. The Technical University of Berlin contributes the battery ageing experiments, the LFP DoE Cycle Ageing Dataset, a battery management system prototype, and the SOC/SOH estimators deployed on its controller. Each storage module must operate autonomously within the flash, RAM, and timing limits of an STM32H753ZI-class microcontroller. Similar constraints arise in mobile systems that cannot rely on continuous cloud access.

Existing embedded studies demonstrate that compact neural SOC or SOH estimators are feasible on microcontrollers and that quantization can reduce their storage requirements [10, 11, 12, 13, 14]. However, these studies generally examine one battery state, one model family, or one deployment platform. They do not provide a common, auditable comparison of structured hidden-unit pruning and recurrent-weight quantization for both stateful SOC and SOH estimators under identical long-horizon replay and hardware measurement conditions. Reproduction is further hindered when scaling parameters, recurrent-state handling, and the exact mixed-precision boundary are not reported.

This work uses the Lithium Iron Phosphate (LFP) DoE Cycle Ageing Dataset [15]. It contains long-term cyclic ageing measurements from multiple LFP cells under full and fractional factorial conditions, with controlled temperature, charge and

discharge rates, depth-of-discharge variations, and interleaved capacity, open-circuit-voltage, and pulse-power check-ups. These data support a paired assessment of rapid SOC dynamics and slowly evolving SOH trajectories.

The objective of this study is to quantify, under identical data and deployment conditions, how structured hidden-unit pruning and recurrent-weight post-training quantization change the accuracy, memory footprint, and inference cost of stateful LSTM + MLP SOC and SOH estimators on an STM32H753ZI. The study makes three contributions:

(i) an auditable path from trained PyTorch models to stateful C inference, including explicit feature scaling, recurrent-state handling, pruning, and mixed-precision quantization.

(ii) a controlled Base–Pruned–Quantized comparison for both SOC and SOH using continuous replay, linked-firmware flash and RAM, host latency, and cycle-counter-measured kernel time.

(iii) an evidence-bounded trade-off analysis that connects architecture-level operation counts and implementation details to long-horizon accuracy, filtering, and application-dependent utility.

The novelty lies in this paired, transparent deployment and compression benchmark rather than in proposing a new recurrent cell or compression algorithm.

2. Related Work and Motivation

2.1. SOC/SOH Estimation

Existing SOC/SOH estimation methods can be grouped into three families [2, 3, 4, 5]. First, *measurement-derived* techniques use Coulomb counting and open-circuit-voltage (OCV) look-up tables during rest periods. They provide inexpensive proxies but suffer from drift, offset accumulation, and a strong dependence on initialisation and periodic recalibration. Second, *model-based* observers combine equivalent-circuit or electrochemical models with Kalman filtering and its variants. These approaches can offer physical interpretability but require careful parameter identification and are sensitive to model mismatch and

ageing-induced parameter drift. Third, *data-driven* methods employ machine-learning regressors, including LSTM networks, Transformers, and hybrid structures, to map sequences of current, voltage, and temperature to SOC and SOH estimates. Key challenges for such models are robustness when operated under conditions that differ from the training data and the careful treatment of normalisation and state handling on embedded targets.

Recent hybrid estimators combine physical or statistical preprocessing with learned correction models. Wang et al. [16] couple online parameter identification with an adaptive radial-basis correction and differential support-vector machine for SOC estimation under dynamic vehicle-oriented tests. For whole-life capacity estimation, Wang et al. [17] combine singular filtering, Gaussian-process regression, and an LSTM to address fast ageing and varying currents. These studies broaden the operating conditions and model structures considered for battery-state estimation. Their primary evidence concerns estimation accuracy, however. Neither reports a paired post-training pruning and quantization study with linked-firmware memory and recurrent microcontroller timing. They are therefore complementary to, rather than substitutes for, the deployment benchmark developed here. Across these families the reported accuracy spans a wide range. Electrochemical or Equivalent Circuit Model (ECM)-based observers with carefully tuned filters often achieve SOC root-mean-square errors (RMSE) well below 1 % of full scale and sub-percent SOH capacity errors on laboratory test cycles [18, 19, 20, 21]. Recent sequence-model approaches based on N-BEATS and Bidirectional LSTMs (BiLSTMs) for sodium-ion cells report SOC Mean Absolute Error (MAE) between roughly 0.1 and 0.4 % [22, 23], whereas TinyML-style SOC estimators deployed on microcontrollers often operate in the 1–3 % error regime [10, 11, 12]. These values provide context rather than a direct ranking because chemistry, data splits, target construction, drive cycles, and error definitions differ between studies. Kalman filters, particle filters, and equivalent-circuit-model observers therefore dominate the classical literature, yet their accuracy can degrade over long operating periods with pronounced current and temperature gradients [19, 21]. Recurrent neural networks, and in

particular LSTM-based regressors, address these limitations by integrating long temporal context and absorbing residual modelling errors [24, 21]. In this work we deliberately restrict the architecture search to a single-layer LSTM with a fully connected Multi-Layer Perceptron (MLP) output layer. LSTMs retain explicit cell states with separate input, forget, and output gates, which made it straightforward to hand-code a numerically stable, stateful kernel in C and to reuse the same backbone for both SOC and SOH. Gated recurrent units (GRUs) or Transformer variants could offer alternative trade-offs between accuracy and cost, but exploring multiple architectures would dilute the focus on the embedded tool flow and is left as future work. Most publications on LSTM-based estimators maximise accuracy with complex architectures and rarely examine how the resulting models behave once deployed on constrained controllers. Even within the LSTM family, sliding-window inference is commonly employed because it delivers high accuracy, but it also requires substantial RAM to buffer multiple past samples. A stateful LSTM that hands over its hidden and cell vectors step by step retains the temporal information without maintaining large windows and therefore forms the first optimisation step in our architecture. The associated handling of hidden states, feature scaling, and numerical stability is rarely described in the literature, which is why this work explicitly targets embedded SOC/SOH deployment.

2.2. Embedded Deployment Strategies

Commercial toolchains such as ONNX Runtime, TensorRT, and STM32Cube.AI support many neural-network deployment workflows [25, 26, 27]. For continuously stateful estimators, however, a generated binary does not always expose recurrent-state handling or the precise placement of mixed-precision scaling operations needed for an implementation audit. Naguib et al. [14] compare accuracy, execution time, flash, and RAM for classical and neural SOC estimators on two NXP microcontrollers. Crocioni et al. [10] deploy compact recurrent capacity estimators on STM32 targets and study 8-bit quantization. More recent TinyML studies evaluate quantized or hardware-accelerated SOC estimators

update, scaling operations, and stored parameter types inspectable and links the model transformations directly to the reported flash, RAM, and timing measurements.

3. Dataset and Preprocessing

3.1. Ageing Experiments and Load Profiles

All experiments rely on the high-resolution LFP DoE Cycle Ageing Dataset [15]. The measurements originate from a design-of-experiments study on commercially available JGCFR18650-1800mAh-3.2V LFP cells [29] conducted at the Technical University of Berlin. Cells were cycled inside a climatic chamber at 25°C while the following factors were systematically varied:

$$C_{\mathrm{chg}} \in [0.5\,\mathrm{C},\ 0.9\,\mathrm{C}], \tag{1}$$

$$C_{\mathrm{dis}} \in [1.0\,\mathrm{C},\ 2.5\,\mathrm{C}], \tag{2}$$

$$\mathrm{DoD} \in [45\%,\ 65\%]. \tag{3}$$

The mixed full/fractional factorial plan distributes fifteen test cells across eight operating-point combinations and includes replication to assess repeatability. Figure 2 shows the low and high levels of all three factors.

Between ageing loops, dedicated check-ups provide capacity tests, open-circuit-voltage sweeps, and hybrid pulse-power characterisation. Stationary-storage profiles are additionally injected to emulate realistic power flows [30]. These profiles originate from residential photovoltaic home-storage applications and contain stochastic load and generation patterns that excite the cell differently from constant-current cycling.

The raw dataset contains uniformly sampled measurements at 1 Hz, including voltage, current, terminal temperature, and auxiliary channels. Partitioned Parquet tables support efficient retrieval of the long ageing sequences. The dataset is published in [15]. The cube notation follows standard DoE conventions as summarised in [31]. Together, Figures 3 and 4 illustrate the diversity of

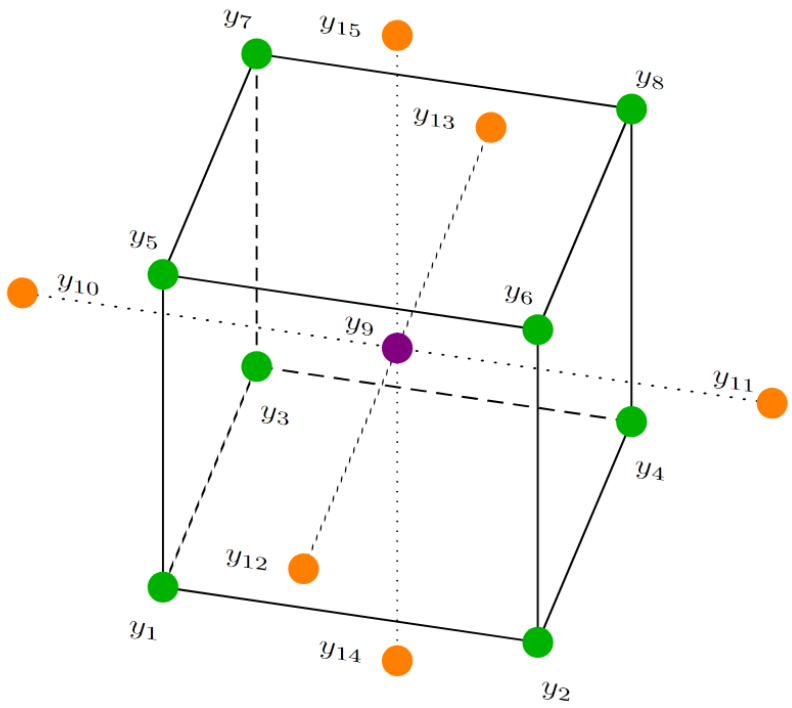

Figure 2: General three-factor design-of-experiments geometry used to structure the LFP ageing campaign. The cube represents the factorial design space, while the generic labels y_1 to y_{15} denote experimental conditions within and around it. The dataset-specific charge rate, discharge rate, and depth-of-discharge levels are described in the accompanying text.

SOC transients and ageing behaviour that the estimators must capture: short-term SOC dynamics are driven by fast current swings and mode changes between profiles, whereas SOH evolves slowly over hundreds of days and depends on the cumulative operating conditions of each cell.

3.2. Feature Engineering

The SOC model consumes the six-dimensional feature vector

$$\mathbf{x}_t^{\mathrm{SOC}} = \left[U(t),\, I(t),\, T(t),\, Q_c(t),\, \tfrac{\mathrm{d}U}{\mathrm{d}t}(t),\, \tfrac{\mathrm{d}I}{\mathrm{d}t}(t)\right], \tag{4}$$

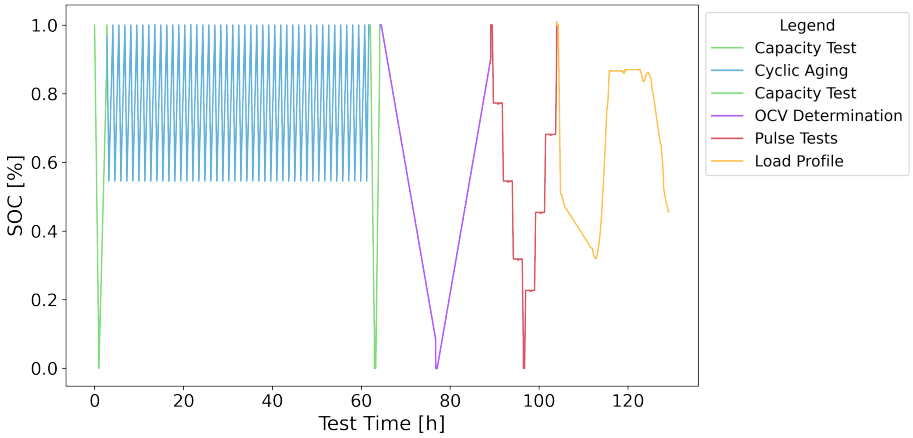

Figure 3: Representative SOC trajectory illustrating selected segments of the experimental protocol used in this study. The plot shows a portion of the cyclic ageing procedure with defined charge and discharge currents and a specified depth-of-discharge (DoD), followed by the diagnostic checkup sequence comprising OCV determination, HPPC pulses, and capacity tests. In addition, the trajectory includes a load profile taken from a stationary battery storage system. Together, these segments capture both controlled laboratory diagnostics and application-oriented operating conditions.

where the audited preprocessing uses timestamp-aware backward differences rather than centred differences. For sample index $k > 0$,

$$\Delta t_k = \max(t_k - t_{k-1}, \varepsilon_t), \qquad \varepsilon_t = 10^{-6}, \tag{5}$$

$$\left.\frac{\mathrm{d}U}{\mathrm{d}t}\right|_k = \frac{U_k - U_{k-1}}{\Delta t_k}, \qquad \left.\frac{\mathrm{d}I}{\mathrm{d}t}\right|_k = \frac{I_k - I_{k-1}}{\Delta t_k}. \tag{6}$$

This causal calculation requires only the preceding voltage/current sample and timestamp and has constant work per update. In the evaluated benchmark, however, derivative and cumulative channels were prepared host-side before replay. Complete six-feature vectors were transmitted via UART, and the firmware applied the same robust-scaling parameters used during model development. Consequently, the reported DWT kernel times exclude sensor acquisition and feature construction. The exact placement of the six-feature scaling operation is documented with the STM32 measurement interval below. Each component is normalised using a robust scaling operation based on the median m and in-

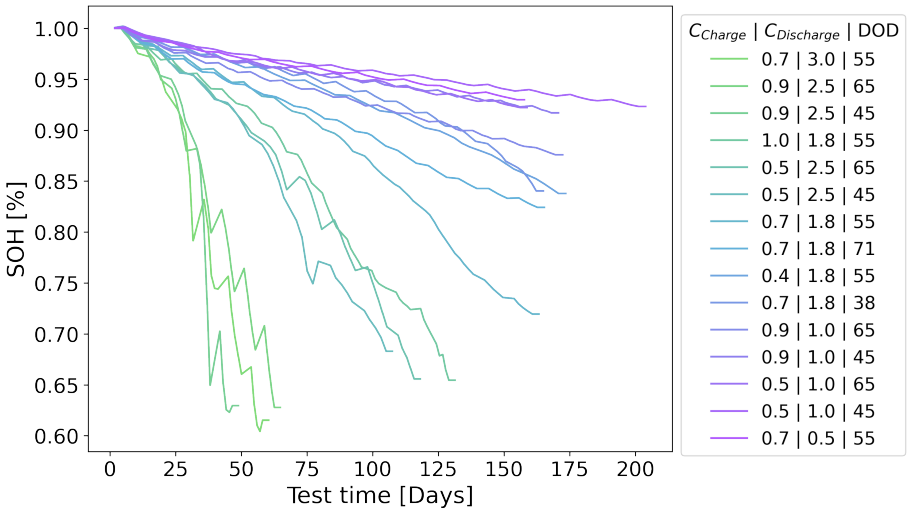

Figure 4: Evolution of the state of health across all LFP cells in the ageing campaign, shown over the total experimental duration. The spread between trajectories reflects the different operating points in the factorial plan (C-rates, depth of discharge, profile mix) and motivates the use of a data-driven SOH estimator that generalises across cells and loading conditions.

terquartile range r:

$$\tilde{x}_{t,k}^{\mathrm{SOC}} = \frac{x_{t,k}^{\mathrm{SOC}} - m_k}{r_k + \varepsilon}, \qquad \varepsilon = 10^{-6}. \tag{7}$$

The same (m_k, r_k) pairs are embedded in the firmware so that host and STM32 inputs are normalised identically. The SOH model uses a slightly different feature stack to capture slow-varying degradation markers:

$$\mathbf{x}_t^{\mathrm{SOH}} = \big[t,\, U(t),\, I(t),\, T(t),\, \mathrm{EFC}(t),\, Q_c(t)\big], \tag{8}$$

where t is the cumulative time stamp and $\mathrm{EFC}(t)$ denotes the equivalent full cycles accrued up to sample t. These channels are likewise robustly normalised, relying on median and interquartile range rather than mean and variance to reduce the influence of outliers in current and voltage measurements [32, 33]. During deployment, the SOC and SOH models run independently on the microcontroller and each produces one estimate per second. In principle, both estimates could be fused more tightly, but this work keeps the models separate to evaluate their resource use and accuracy independently.

3.3. Reference SOC and SOH targets

The ground-truth SOC trajectory is derived from Coulomb counting with drift compensation. Denoting the apparent cell capacity after a given check-up by C_{app}, the SOC inside a charge-discharge segment is computed as

$$\mathrm{SOC}(t) = \mathrm{SOC}(t_0) - \frac{1}{C_{\mathrm{app}}} \int_{t_0}^{t} I(\tau)\, \mathrm{d}\tau, \tag{9}$$

where the integral is reset to SOC = 1 after each constant-voltage (CV) termination. The reset mitigates the bias that would otherwise accumulate due to capacity fade, yet the long-term traces still reveal gradual changes in the SOC envelope as C_{app} decreases. State of health is obtained from the periodic capacity check-ups through

$$\mathrm{SOH}_k = \frac{C_k}{C_{\mathrm{nom}}}, \tag{10}$$

with C_k representing the measured discharge capacity during the k-th reference cycle (comprising capacity test, open-circuit-voltage sweep, and hybrid pulse-power characterisation) relative to the nominal capacity C_{nom}. These targets highlight the need for a data-driven estimator that remains accurate despite the evolving capacity landscape, which motivates the use of recurrent neural networks for both SOC and SOH predictions. The reference SOC and SOH traces are therefore offline constructs. They rely on information that is not available in deployment, such as long rest periods for OCV stabilisation and capacity from periodic full cycles, and they assume perfect initialisation and recalibration. These constructions follow common practice in battery management, where SOC is obtained by Coulomb counting and SOH is expressed as the ratio between present and nominal capacity [34, 2, 3]. On the STM32 the estimators operate purely on the streamed measurements and their internal states. The reference trajectories only serve as ground truth for training and evaluation.

3.4. Streaming inference setup

Because the SOC and SOH estimators are stateful, the LSTM hidden and cell states are continuously forwarded from one sample to the next along the full

input stream. At each sampling instant the current feature vector is processed together with the previous hidden and cell states, which yields the new states and the corresponding SOC or SOH estimate. We therefore operate the models in a purely step-by-step streaming regime without forming sliding input windows. On the host the full normalised sequence from the LFP DoE Cycle Ageing Dataset is processed once in this fashion, and the resulting SOC and SOH estimates are aligned with the reference trajectories. On the STM32 firmware the same step-wise LSTM implementation is used, driven by the UART feature stream, so that host and embedded benchmarks reflect identical inference conditions. The streaming dashboards in Figs. 8 and 11 illustrate a complete run for one representative cell.

3.5. SOH prediction post-processing

The SOH trajectories in the final dashboard use two sequential causal filtering stages. First, each raw model output $\hat{y}_t$ is multiplicatively aligned with the initial SOH label, $\hat{y}_t^{(0)} = (y_0/\hat{y}_0)\hat{y}_t$. Stage 1 limits the change of this calibrated prediction relative to the preceding stage output to

$$\ell_t = \min\left(10^{-4}|y_{t-1}^{(1)}|, 10^{-5}\right), \tag{11}$$

and subsequently applies an exponential moving average (EMA) with $\alpha_1 = 0.02$. The limiter is symmetric, so positive and negative proposal changes are both restricted to $\pm\ell_t$. Stage 2 consumes the complete stage-1 trajectory, applies another EMA with $\alpha_2 = 10^{-6}$, and limits only downward changes according to

$$y_t^{(2)} = \max\left(e_t^{(2)}, y_{t-1}^{(2)} - 2\times 10^{-8}\right), \qquad e_t^{(2)} = (1-\alpha_2)e_{t-1}^{(2)} + \alpha_2 y_t^{(1)}. \tag{12}$$

Thus, the curve in Fig. 11 is not obtained by applying α_2 directly to an unfiltered network output.

For an EMA sampled at interval Δt, the equivalent time constant and cutoff frequency are

$$\tau = -\frac{\Delta t}{\ln(1-\alpha)}, \qquad f_c = \frac{1}{2\pi\tau}. \tag{13}$$

At the dataset rate of 1 Hz, stage 1 has $\tau = 49.5\,\mathrm{s}$, $f_\mathrm{c} = 3.22\,\mathrm{mHz}$, and an EMA-only 90 % step-response time of approximately 114 s. For stage 2, τ is approximately 11.57 days, $f_\mathrm{c} = 1.59\times10^{-7}$ Hz, and the corresponding 90 % time is 26.65 days. These frequencies characterise only the linear EMA components. The nonlinear rate limiters have no single cutoff frequency. The very strong second stage can reduce error against slowly varying labels while introducing substantial response delay, which is evaluated separately in Appendix A.3.

4. Model Architecture and Baseline Performance

4.1. LSTM architecture in PyTorch

Both SOC and SOH estimators employ the same recurrent equation set but differ in size according to their training releases. The SOC configuration uses $H = 64$ hidden units and an MLP output layer of width 64, whereas the SOH configuration increases both dimensions to 128 to cope with the longer temporal context. LSTM gates i_t, f_t, o_t and candidate state g_t follow the standard formulation used in modern RNN surveys [35]. All models are implemented in PyTorch [36]. For one streaming update with input dimension $D = 6$, LSTM hidden size H, and fixed MLP width M, the matrix products require

$$N_\mathrm{MAC}(H) = 4H(D + H) + HM + M, \tag{14}$$

where one multiply–accumulate (MAC) is counted per weight. Bias additions, nonlinearities, indexing, and memory transfers are excluded. For a hidden-unit pruning fraction p, the retained hidden size is $H_p = (1-p)H$. Substitution into Eq. (14) gives the relative analytical MAC reduction

$$\begin{aligned} R_\mathrm{MAC}(H,p) &= 1 - \frac{N_\mathrm{MAC}((1-p)H)}{N_\mathrm{MAC}(H)} \\ &= \frac{4(2p-p^2)H^2 + p(4D+M)H}{4H^2 + (4D+M)H + M}, \\ \lim_{H\to\infty} R_\mathrm{MAC}(H,p) &= 2p - p^2. \end{aligned}$$

The quadratic recurrent term is therefore reduced by $2p - p^2$, the linear input and MLP terms by p, and the fixed output term is unchanged. The finite-model reduction is a weighted combination of these contributions and remains below the large-H limit. For the evaluated fraction $p = 0.30$, this limit is 51 %. The implemented integer dimensions give 22,080 to 12,124 MACs for SOC ($64 \rightarrow 45$), a 45.1 % reduction, and 85,120 to 46,208 MACs for SOH ($128 \rightarrow 90$), a 45.7 % reduction. Appendix A.6 visualises both this operating point and the general dependence on p. The relation makes the computational effect of a selected hidden-size reduction predictable before deployment, while hardware measurements translate the analytical reduction into platform-specific flash, latency, and energy changes.

$$
\begin{aligned}
\mathbf{z}_t &= [\mathbf{x}_t, \mathbf{h}_{t-1}], \\
\mathbf{i}_t &= \sigma(W_i \mathbf{z}_t + \mathbf{b}_i), \quad \mathbf{f}_t = \sigma(W_f \mathbf{z}_t + \mathbf{b}_f), \\
\mathbf{o}_t &= \sigma(W_o \mathbf{z}_t + \mathbf{b}_o), \quad \mathbf{g}_t = \tanh(W_g \mathbf{z}_t + \mathbf{b}_g), \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).
\end{aligned}
\tag{15}
$$

Equations (14) and the recurrent update above define the common Base architecture used for both tasks. Pruning changes its hidden dimension and parameter submatrices, whereas quantization changes the stored representation of selected recurrent weights. Neither transformation changes the input features, targets, or streaming state-update order.

Both SOC and SOH models are trained with a mean-squared-error loss between the prediction and the corresponding reference at each training sample. During validation we additionally report mean absolute error (MAE), root-mean-square error (RMSE), and R^2. The trained parameters, configuration files, and optimiser statistics are retained to enable deterministic replay of both models.

4.2. Training Notes

Training is carried out on a GPU node equipped with NVIDIA A100-class accelerators using mixed-precision (FP16) arithmetic. Both models use the

AdamW optimiser with cosine-annealing warm-restart scheduling, gradient clipping, and mini-batch training [37, 38, 39, 40]. The SOC configuration uses a learning rate of 1.5×10^{-4}, a batch size of 512, and sequence chunks of length 2024. The corresponding SOH values are 2.0×10^{-4}, 128, and 2048.

Deployment operates the LSTM in stateful, step-by-step streaming mode, whereas training uses truncated backpropagation through time over these chunks. Hidden and cell states are propagated within every chunk in the same temporal order as during deployment. Chunking permits randomised segment sampling and tractable gradient computation without changing the recurrent update equations.

4.3. Baseline Accuracy

On the workstation the SOC and SOH base models are first evaluated in streaming mode on the full ageing trajectory of one representative LFP cell from the LFP DoE Cycle Ageing Dataset [15], yielding baseline MAE and RMSE values for both tasks. These Python replays operate on the original PyTorch implementations and provide the numerical reference for the compression experiments. The corresponding trained models are pruned, quantized, and exported to STM32 firmware, where identical feature streams are replayed to measure accuracy, flash and RAM usage, system latency, and on-device inference time. A timing-derived energy proxy is calculated as described in Section 7. Figure 5 summarises this training-to-firmware benchmarking pipeline.

5. Pruning

Pruning reduces the size and computational cost of a neural network by eliminating parameters or whole units that contribute little to the overall mapping, while keeping the input-output behaviour as close as possible to the dense baseline [41, 42]. In our setting we apply structured pruning at the level of complete LSTM hidden units so that the resulting model remains well suited for efficient C implementations on the microcontroller.

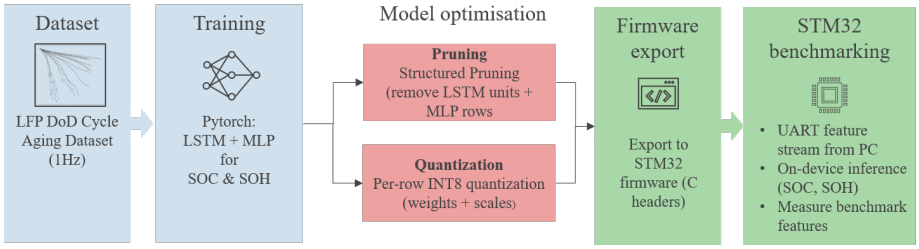

Figure 5: End-to-end pipeline from dataset to embedded benchmarking. The LFP DoE Cycle Ageing Dataset feeds PyTorch training on a GPU workstation. The resulting SOC and SOH LSTM + MLP models are pruned and quantized, exported as C headers, and deployed on an STM32H753ZI board, where flash/RAM usage, inference time, and latency are measured under streaming replay. Estimated energy per inference is derived from kernel time under a constant board-power assumption.

5.1. Structured magnitude pruning

Structured magnitude pruning removes the lowest-saliency LSTM units until only 45 hidden channels remain in the SOC model (30% sparsity), with analogous settings for the SOH experiments. Reducing the state dimension is a prerequisite for fitting the estimators into the flash and SRAM budgets of the target STM32-based BMS and for shortening the on-device inference time, inspired by structured sparsity approaches for deep and recurrent networks [43, 44, 45]. In analogy to the group-lasso style regularisers proposed for LSTMs in [43, 44], which aggregate weights across gates via L2 norms, we define a saliency score for each hidden unit h that combines all incoming weights from the different gates:

$$s_h = \sum_{g\in\{i,f,o,g\}} \left(\|W_{\mathrm{ih}}^{(g)}[h,:]\|_2 + \|W_{\mathrm{hh}}^{(g)}[h,:]\|_2 \right). \tag{16}$$

Units are ranked by s_h, and the H' highest-scoring indices form the retained set $\mathcal{R}$. For each gate g, structured pruning constructs the reduced matrices

$$\begin{aligned} \widetilde{W}_{\mathrm{ih}}^{(g)} &= W_{\mathrm{ih}}^{(g)}[\mathcal{R},:], \\ \widetilde{W}_{\mathrm{hh}}^{(g)} &= W_{\mathrm{hh}}^{(g)}[\mathcal{R},\mathcal{R}], \\ \widetilde{W}_{\mathrm{MLP},1} &= W_{\mathrm{MLP},1}[:,\mathcal{R}]. \end{aligned} \tag{17}$$

Thus the same index set removes gate rows, recurrent columns, and the corresponding inputs of the first MLP layer. The reduced tensors remain dense. The computational saving comes from the smaller dimension rather than sparse indexing.

The final SOC deployment uses $H' = 45$ instead of 64 hidden units, while SOH uses $H' = 90$ instead of 128. Both transformations remove approximately 30 % of the recurrent channels. The fraction $p = 0.30$ was selected as a single moderate operating point that provides a substantial dimensional reduction while retaining useful estimator accuracy. It was not obtained from an exhaustive pruning-rate sweep and is not claimed to be optimal. Establishing an optimal fraction would require separately pruned and fine-tuned models for each candidate rate and is reserved for a pruning-focused investigation. This is a one-shot structured operation followed by brief post-pruning fine-tuning. No iterative pruning schedule or pruning-aware training is used. The L2 aggregation gives a gate-consistent ranking that can be calculated directly from the model weights and mapped to dense C arrays. Appendix A.1 reports the resulting SOC saliency ranking and recurrent-weight distributions. These diagnostics describe the resulting weight selection but do not establish a causal regularisation effect. The criterion is selected because it combines all four gate rows for one recurrent unit and maps directly to removal of dense rows, recurrent columns, and the associated MLP input. It therefore provides a reproducible, gate-consistent ranking that translates directly into a smaller dense kernel. Gradient sensitivity would additionally use training data and backpropagation, while activation-based ranking would use a representative calibration stream and a temporal aggregation rule. A future comparison with these data-dependent alternatives can determine whether their additional information further improves the accuracy–efficiency trade-off. Appendix A.7 summarises the method properties.

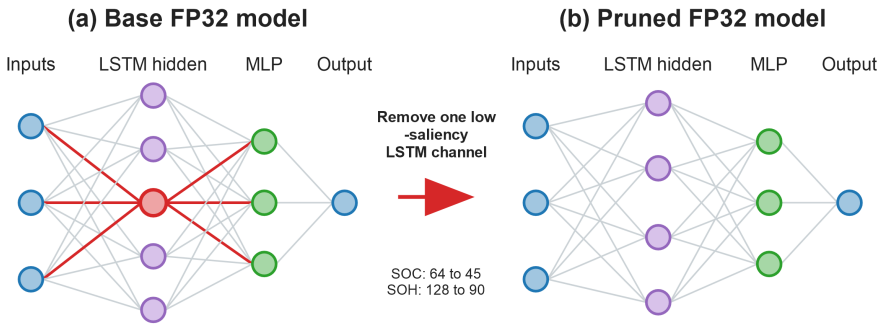

Figure 6: Structured hidden-channel pruning used in this study. Panel (a) marks one low-saliency LSTM channel and its incident connections in red. Panel (b) shows the resulting dense network after removing the corresponding gate rows, recurrent columns, and MLP input column according to Eq. (17). The final SOC and SOH hidden dimensions are shown between the panels.

5.2. Resulting pruned models

The resulting pruned SOC and SOH models still run in FP32 but reduce flash consumption and runtime considerably once compiled for STM32. Section 7 summarises the corresponding latency and memory savings. These pruned variants are evaluated alongside their base and quantized counterparts throughout the results section.

6. Quantization

Quantization complements pruning by reducing the numerical precision of selected weights while keeping the network topology unchanged. We apply post-training, weight-only quantization to the LSTM input-to-hidden matrix W_{ih} and hidden-to-hidden matrix W_{hh}. The resulting matrices are stored as 8-bit integers together with one FP32 scale per row. Biases, activations, recurrent hidden and cell states, and the MLP remain FP32. This design concentrates the storage reduction on the dominant recurrent matrices while retaining a mixed-precision computation path.

6.1. Per-row symmetric quantization

The FP32 matrices $W_{\text{ih}}^{\text{fp32}}$ and $W_{\text{hh}}^{\text{fp32}}$ map the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, respectively, to the four stacked LSTM gates. Quantization is applied independently to every row of both matrices. For $W^{\text{fp32}} \in \{W_{\text{ih}}^{\text{fp32}}, W_{\text{hh}}^{\text{fp32}}\}$, row r is converted according to

$$s_r = \max\left(\frac{\max_k |(W^{\text{fp32}})_{r,k}|}{127}, \varepsilon_q\right), \tag{18}$$

$$(W^{\text{int8}})_{r,k} = \text{clip}\left(\text{round}\left(\frac{(W^{\text{fp32}})_{r,k}}{s_r}\right), -127, 127\right), \tag{19}$$

$$(\hat{W})_{r,k} = s_r (W^{\text{int8}})_{r,k}. \tag{20}$$

Here, W^{int8} denotes the stored integer matrix and $\hat{W}$ its effective floating-point reconstruction. The lower bound ε_q prevents division by zero for an all-zero row. Although the INT8 storage type provides 256 machine codes, the implementation leaves -128 unused and employs the symmetric 255-level set $\{-127, \ldots, 127\}$. This keeps zero exactly representable and assigns equal positive and negative ranges. With the max-absolute scale in Eq. (18), finite row weights do not clip, and nearest-integer rounding gives the element-wise reconstruction bound

$$\left|(W^{\text{fp32}})_{r,k} - (\hat{W})_{r,k}\right| \leq \frac{s_r}{2}. \tag{21}$$

Per-row scaling gives small-magnitude rows a finer step size than one global scale, following standard uniform post-training quantization practice [46, 47].

6.2. Quantization details

Using the stored matrices $W_{\text{ih}}^{\text{int8}}$ and $W_{\text{hh}}^{\text{int8}}$, the complete gate pre-activation is

$$\mathbf{a}_t = \mathbf{s}_{\text{ih}} \odot \left(W_{\text{ih}}^{\text{int8}} \mathbf{x}_t\right) + \mathbf{s}_{\text{hh}} \odot \left(W_{\text{hh}}^{\text{int8}} \mathbf{h}_{t-1}\right) + \mathbf{b}_{\text{fp32}}. \tag{22}$$

The vectors $\mathbf{s}_{\text{ih}}$ and $\mathbf{s}_{\text{hh}}$ apply the corresponding FP32 scale to each gate row. Equation (22) is the compact matrix form of the C implementation. In the current kernel, each accumulation term is evaluated as an FP32 input or hidden-state value multiplied by an INT8 weight cast to FP32 and by its row scale. The

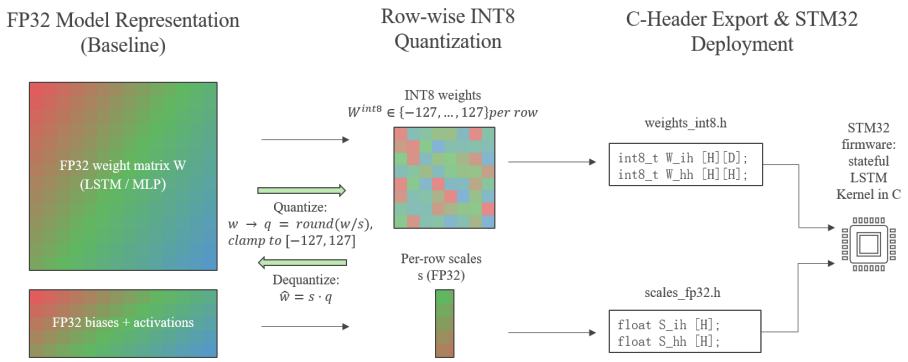

Figure 7: FP32-to-INT8 quantization and embedded deployment. The recurrent FP32 weight matrices are mapped row by row to signed INT8 codes in the symmetric set $\{-127, \ldots, 127\}$ together with one FP32 scale per row. The integer recurrent weights and scale vectors are exported as C headers and combined with FP32 biases, activations, recurrent states, and MLP operations in the STM32 kernel.

recurrent weights therefore occupy INT8 storage, while the gate products and accumulations remain FP32. A built-in streaming comparison validates the exported gate computations against the source model and typically yields MAE differences below 10^{-4}.

The recurrent matrices account for approximately 80 % of the raw Base model constants in both tasks. Targeting these matrices concentrates the storage reduction on the dominant constants while retaining FP32 biases, activations, recurrent states, and the smaller MLP path. This creates a transparent mixed-precision configuration that reduces persistent storage without changing the recurrent state representation. Quantizing activations and states could extend the approach toward additional RAM and latency reductions and is considered as a subsequent optimisation step. Appendix A.8 documents the precision boundary and model-constant composition.

6.3. Export for STM32

The exported headers are integrated into separate C firmware projects for the stateful SOC and SOH estimators. Each project combines the LSTM and MLP kernels with UART telemetry. A small driver library provides build files

and flashing scripts so that identical measurement binaries can be rebuilt on other machines. This explicit tool chain, from training through pruning and quantization to firmware, exposes the numerical operations and records the build settings used for the resource measurements in Section 7.

7. Benchmark Methodology

This section describes how we benchmark the SOC and SOH models on both the host PC and an STM32 Nucleo-144 development board with an STM32H753ZI MCU [48, 14]. The microcontroller provides 2 MB of on-chip Flash and 1 MB of SRAM, which constitute the budget for firmware code, model parameters, and runtime buffers. Starting from the trained LSTM + MLP models, we generate three embedded variants (full-precision baseline, pruned model, and quantized model) and evaluate them on the *LFP DoE Cycle Ageing Dataset*. The benchmark combines long-horizon streaming simulations with hardware-in-the-loop measurements and is fully scripted to ensure reproducible accuracy and resource evaluations.

On the host side, the complete reference ageing sequence is replayed in Python using the same streaming inference functions that are later exported to C. The SOC and SOH estimators are evaluated in continuous time without resetting their recurrent states, and error metrics (MAE, RMSE, R^2), error histograms, and parity plots are computed from the resulting trajectories. These simulations provide a high-precision reference for the embedded experiments and are used to assess how pruning and quantization affect the dynamics of the baseline models. To test whether deviations grow over time, the full-stream trajectories are divided into ten consecutive, equal-count segments. Target MAE and P95 error are computed in every segment, together with the mean absolute deviation of each compressed variant from the Base output. Appendix A.2 reports this temporal analysis.

For the embedded benchmarks the base, pruned, and quantized SOC/SOH models are embedded into dedicated firmware projects for the STM32H753ZI.

Because STM32Cube AI does not natively support stateful LSTM layers, the recurrent update equations, post-processing, and UART diagnostics are implemented manually in C. Identical stimulus streams are then injected via UART, allowing a direct comparison between host-side and on-device behaviour under the same operating conditions.

7.1. Limited software-level fault sensitivity

Two software-level sensitivity analyses compare the Base, Pruned, and Quantized variants under temporary output-report loss and transient input corruption. The first analysis represents unavailable estimator reports while the underlying inference trajectory continues without interruption. Randomly selected reports at rates of 1, 5, and 10 % are replaced by the most recent available prediction using a hold-last strategy. The same strategy is applied to ten deterministic reporting gaps for each duration of 60, 600, and 3600 consecutive samples, corresponding to 1, 10, and 60 min at the 1-Hz sampling rate. The controlled scope of these analyses isolates output availability and input-buffer corruption from faults in model parameters, activations, recurrent-state memory, communication, and MCU memory.

The second analysis evaluates how a one-sample input-buffer fault propagates through the recurrent state. Each exported C model processes a continuous 8000-sample segment from the evaluation trajectory of one representative LFP cell at 1 Hz. Thirty independent events are distributed over this sequence after a clean warm-up of at least 2048 samples, with ten events each for voltage, current, and temperature. At every event, the disturbed branch starts from the same recurrent state as the clean branch. FP32 bits are numbered from zero at the least significant bit. Bit 22 is therefore the highest-order bit of the 23-bit fraction field. Flipping this bit produces a pronounced but finite change in the selected input value while leaving its sign and exponent unchanged. The bit is flipped for the event sample only. The model then receives 60 subsequent unmodified inputs, corresponding to a 60-s recovery horizon, while its recurrent state continues without a reset.

For event e, $k = 0$ denotes the corrupted sample and $k = 1, \ldots, 60$ the subsequent clean samples. The resulting evaluation window contains 61 samples, while $k = 60$ lies 60 s after the fault because consecutive samples are separated by 1 s. The fault-induced deviation is $d_{e,k} = 100|\hat{y}_{e,k}^{\text{fault}} - \hat{y}_{e,k}^{\text{clean}}|$ in percentage points. Peak deviation and residual deviation after 60 s quantify the transient response independently of the model's pre-existing target error. Accuracy is evaluated over the same window using

$$\Delta\text{MAE}_{61,e} = \frac{100}{61} \sum_{k=0}^{60} \left(\left| \hat{y}_{e,k}^{\text{fault}} - y_{e,k} \right| - \left| \hat{y}_{e,k}^{\text{clean}} - y_{e,k} \right| \right). \tag{23}$$

A positive value indicates that the bit flip increases target MAE, while a negative value means that the disturbed prediction happens to lie closer to the target. Recovery is the first time after the peak at which $d_{e,k}$ remains below $\max(0.1 d_{e,\text{peak}}, 10^{-4}\ \text{pp})$ for five consecutive samples. Together, these metrics quantify the immediate output deviation, the change in target accuracy, and the decay of the disturbance through the recurrent state.

7.2. STM32 Measurements

On the STM32 Nucleo-144 development board, dedicated SOC and SOH firmware images execute the embedded estimators while host-side scripts handle stimulus injection and UART logging. Recorded feature streams are transmitted to the microcontroller, which runs the stateful LSTM + MLP and returns predictions with timing diagnostics.

Each sample produces two time stamps. Host latency measures the delay between sending an input vector and receiving its estimate. On-device inference time is measured with the data watchpoint and trace (DWT) cycle counter around the LSTM and MLP kernels. Reported values are averages over 10,000 streaming inferences. The timed interval includes instructions and memory accesses executed by these kernels but excludes UART transfer, host processing, and host-side feature construction.

In the SOC projects, six-feature robust scaling lies inside the timed estimator call. The SOH Base and Pruned projects apply scaling immediately before the

DWT interval, whereas the Quantized SOH forward function includes it. This small affine preprocessing cost was not profiled separately and is absent from the static operation counts. All inspected benchmark projects were compiled with GCC at `-O0`, using the hard-float ABI and Cortex-M7 FPv5 floating-point unit. The measurements therefore characterise auditable, unoptimised reference kernels rather than production-optimised integer deployments.

7.3. Key Performance Indicators

For normalised target y_n and prediction $\hat{y}_n$, the signed error is expressed in percentage points as $e_n = 100(\hat{y}_n - y_n)$. The reported accuracy indicators are

$$\mathrm{MAE} = \frac{1}{N}\sum_{n=1}^{N}|e_n|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}e_n^2}, \tag{24}$$

$$\mathrm{P95} = Q_{0.95}\left(\{|e_n|\}_{n=1}^{N}\right), \tag{25}$$

where $Q_{0.95}$ is the empirical 95th percentile. MAE and RMSE use all samples, whereas P95 identifies the absolute-error threshold below which 95 % of samples lie.

For each deployment we additionally report average host latency, on-device inference time, flash and SRAM usage, and an estimated energy-per-inference proxy. Tables 1 and 2 focus on these STM32 resource and timing indicators. Latency is the end-to-end UART delay, whereas inference time covers only the DWT-measured kernel interval. RAM combines statically allocated data and BSS sections with maximum observed streaming stack usage.

Energy was not measured separately for each model. Instead, we use

$$E_{\mathrm{est}} = P_{\mathrm{assumed}}\, t_{\mathrm{inf}}, \qquad P_{\mathrm{assumed}} = 0.5\,\mathrm{W}, \tag{26}$$

where 0.5 W is a representative average-power assumption for the complete development board. Consequently, the percentage change in E_{est} is identical to the percentage change in inference time. The proxy does not separate core computation, flash access, SRAM access, voltage-regulator losses, or other board

Table 1: Engineering KPIs for SOC models on STM32H753ZI (full-stream averages). Energy is estimated from measured inference time under an assumed constant board power of 0.5 W.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	E_{est} [mJ]
Base FP32	12.70	1.40	105.32	4.93	0.70
Pruned FP32	12.20 (-3.9%)	0.80 (-42.9%)	62.27 (-40.9%)	4.03 (-18.3%)	0.40 (-42.9%)
Quant INT8	18.48 (+45.5%)	6.99 (+399%)	52.48 (-50.2%)	3.96 (-19.7%)	3.49 (+399%)

Table 2: Engineering KPIs for SOH models on STM32H753ZI (full-stream averages). Energy is estimated from measured inference time under an assumed constant board power of 0.5 W.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	E_{est} [mJ]
Base FP32	34.88	22.73	335.00	8.69	11.37
Pruned FP32	24.69 (-29.2%)	12.72 (-44.0%)	182.41 (-45.5%)	6.96 (-19.9%)	6.36 (-44.0%)
Quant INT8	41.23 (+18.2%)	29.21 (+28.5%)	138.00 (-58.8%)	6.70 (-22.9%)	14.60 (+28.5%)

consumers and cannot determine whether flash access dominates the actual energy consumption. An analogous set of measurements was recorded for the SOH firmware deployments. Table 2 lists the corresponding indicators.

The KPIs in Tables 1 and 2 highlight the trade-offs introduced by pruning and quantization. For SOC, pruning reduces the flash footprint by roughly 41 % and inference time by about 43 %, while latency remains dominated by UART transfer. Quantization approximately halves flash usage, but per-weight scaling and integer–float conversions in the mixed-precision kernels increase on-device inference time by a factor of four relative to Base. The SOH deployments show the same qualitative behaviour. Pruning saves about 46 % of flash and 44 % of inference time, whereas Quantized attains the smallest flash footprint but incurs a 29 % timing increase. Under the constant-power assumption, the estimated energy proxy follows these timing changes exactly. It is not independent evidence of board-level energy efficiency.

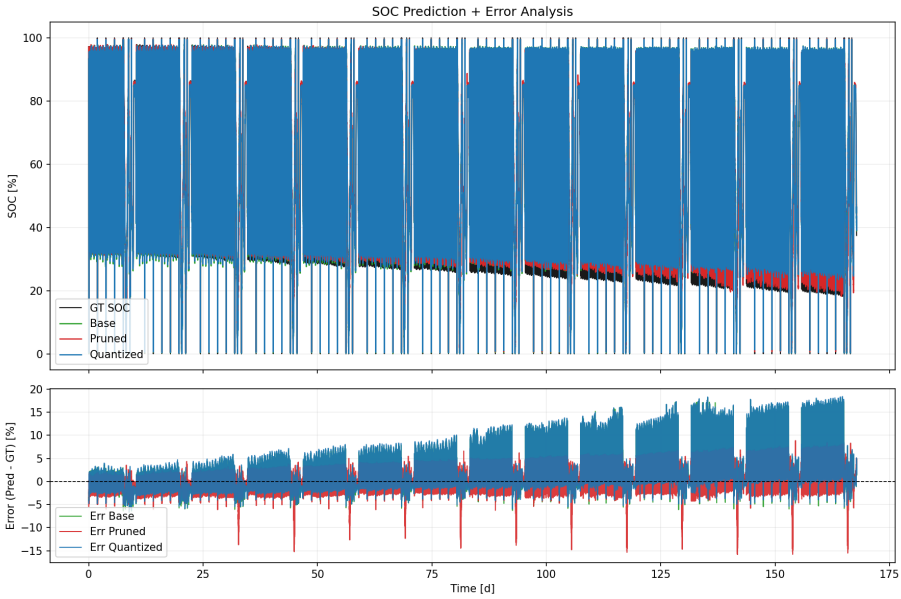

Figure 8: Streaming SOC dashboard on the full ageing trajectory. The black line represents the reference SOC, while the coloured lines show the predictions of the Base (green), Pruned (red), and Quantized (blue) models. The lower sub-panel displays the instantaneous prediction error.

8. Results

The evaluation compares the full-precision Base, Pruned, and Quantized SOC and SOH estimators under the same continuous streaming replay of the LFP DoE Cycle Ageing Dataset. The following sections separate prediction accuracy, temporal behaviour, embedded resource use, and the resulting multi-objective trade-offs.

8.1. Streaming Prediction Accuracy

The primary metric for any battery state estimator is its ability to track the ground truth over extended periods without divergence. Figures 8 and 11 illustrate the streaming performance of the SOC and SOH models over the complete test duration. Across the complete SOC replay, the Base, Pruned, and Quantized variants obtain MAE values of 2.685, 2.338, and 2.791 percentage points,

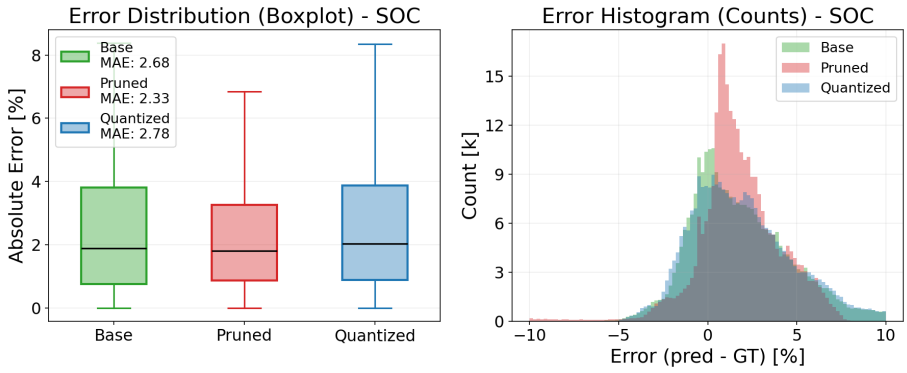

Figure 9: SOC error distributions for Base, Pruned, and Quantized. The left panel compares absolute errors and aggregate MAE. The right panel shows signed errors, $100(\hat{y} - y)$, so that distribution shifts and tails remain visible.

respectively. The Quantized trace often overlaps the Base trace visually, whereas the Pruned model has the lowest aggregate target MAE despite a larger model-to-Base deviation in some parts of the sequence. These results confirm that post-training weight quantization preserves useful recurrent behaviour throughout the evaluated stream. Replays of additional operating sequences can extend this comparison to a wider range of temporal conditions.

The SOH estimator faces a different challenge: it must infer slow capacity degradation from sparse check-up events and continuous monitoring. In the plotted trajectories the discrete capacity tests have been removed from the estimator input, while the remaining check-up blocks (OCV sweeps, HPPC pulses, and profile segments) are retained. The plateau-like sections of the reference curve therefore correspond to these diagnostic intervals. The dashboard applies the complete two-stage causal post-processing chain from Section 3.5: initial alignment, stage 1 with $\alpha_1 = 0.02$ and a symmetric limiter, followed by stage 2 with $\alpha_2 = 10^{-6}$ and a downward limiter. The resulting smoothing suppresses short-term variation but introduces a substantial accuracy–delay trade-off.

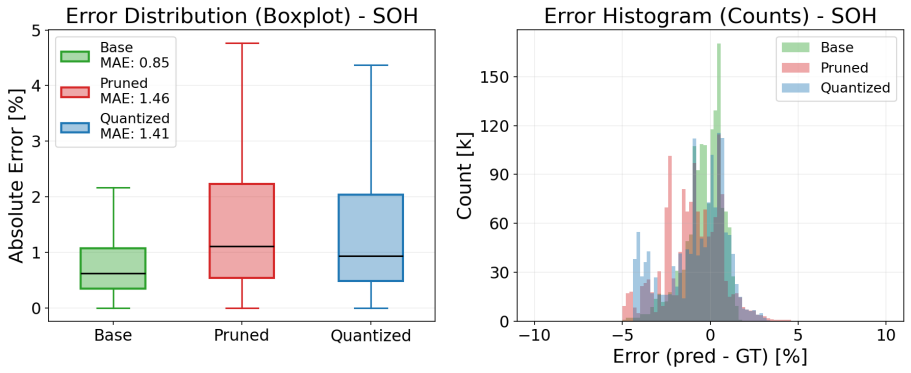

Figure 10: SOH error analysis: MAE distribution (left) and error histogram (right). The three variants retain similar central error ranges, while their tails and aggregate MAE values differ.

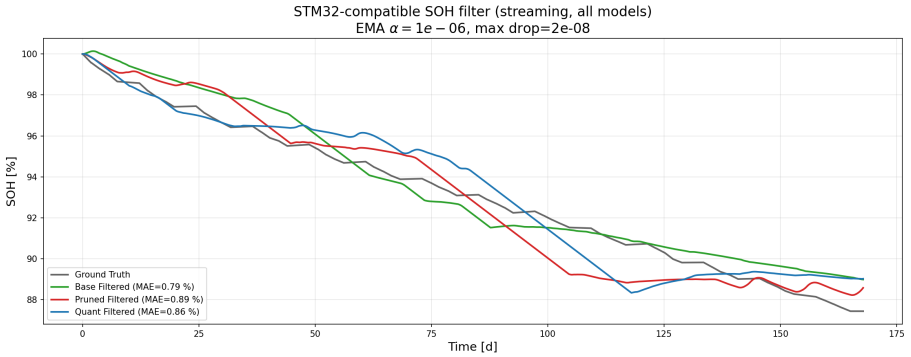

Figure 11: Streaming SOH dashboard with the complete two-stage causal post-processing chain from Section 3.5. The top panel shows the reference SOH and the outputs of the Base (green), Pruned (red), and Quantized (blue) models. The bottom panel depicts the corresponding filtered prediction errors.

8.2. Temporal stability and filter interaction

The ten-segment analysis in Appendix A.2 retains the natural temporal order and does not reset recurrent states between segments. SOC target error rises toward the later part of the sequence for every variant: for example, Base MAE increases from 1.116 to 4.538 percentage points and Quantized MAE from 1.234 to 4.600 percentage points between the first and final tenths. However,

the Quantized-to-Base mean absolute deviation changes from 0.332 to 0.271 percentage points over the same interval and therefore shows no quantization-specific monotonic accumulation. The Pruned-to-Base deviation increases from 0.924 to 2.592 percentage points even though Pruned has the lowest aggregate target MAE. SOH window errors fluctuate with the ageing phase, and neither compressed variant exhibits monotonic model-to-Base divergence across all ten segments. The analysis therefore demonstrates stable relative behaviour throughout the complete evaluated replay and provides a basis for extending validation to further operating sequences.

Appendix A.3 compares unfiltered, stage-1, and final two-stage SOH outputs for the three exported C models on a common input sequence. Stage 1 gives MAE values of 1.677, 1.644, and 1.926 percentage points for Base, Pruned, and Quantized, respectively. Applying stage 2 to the stage-1 output gives final MAE values of 1.476, 1.695, and 1.028 percentage points. The identical input and processing order reveal how filtering changes both the absolute error and the ranking of the model variants. Combining raw and filtered accuracy with the filter response time therefore provides a more complete assessment of the estimator dynamics. The final EMA has an EMA-only 90 % response time of 26.65 days at 1 Hz.

8.3. Error Distribution Analysis

To quantify the reliability of the estimators, we analyse the statistical distribution of the prediction errors. Figures 9 and 10 present the Mean Absolute Error (MAE) boxplots and the error histograms for both tasks. The SOC error distribution is narrow and approximately centred around zero. The evaluated Pruned model attains the smallest aggregate MAE, while the Quantized model remains close to the Base result with a smaller recurrent-weight footprint. Appendix A.1 shows that the 19 removed SOC units occupy the lowest positions of the defined L2 saliency ranking and compares the central recurrent-weight distributions before and after pruning. This confirms that the implemented criterion removes the intended low-saliency channels and identifies Pruned as the

most accurate variant for the evaluated model and data partition. Repeated training runs can determine the statistical consistency of this advantage without attributing it to a specific regularisation mechanism. For SOH, the error distribution is broader, which is inherent to the difficulty of estimating capacity fade from operational data. However, the histograms show that the bulk of the errors are still contained within a useful tolerance band (typically $\pm 2\%$), making the embedded estimator a viable tool for onboard diagnostics.

8.4. Transient Response and Zoomed Analysis

Global metrics often hide local failures. To inspect the transient behaviour, we zoom into a high-dynamic pulse discharge phase (Fig. 12) and into a representative check-up sequence (Fig. 13).

In the pulse region (Fig. 12), the models must react to rapid current changes. The quantized model follows the ground truth transients with high fidelity, capturing the voltage recovery phases accurately. The pruned model, having fewer parameters to model the complex electrochemical relaxation, tends to smooth out these sharp features slightly. During the check-up phase (Fig. 13), where the cell undergoes full charge/discharge cycles, all models converge well, demonstrating stability over long integration horizons.

8.5. Resource Efficiency and Latency

Figures 14 and 15 summarise the flash memory, RAM usage, parameter counts, and latency distributions for the SOC and SOH deployments on the STM32H7 microcontroller. For all deployments the model complexity is first characterised on the workstation side by counting parameters in each variant (base, pruned, quantized). Assuming four bytes per weight for floating point representations and one byte per weight for INT8, this directly yields an estimate of the flash footprint attributable to the neural network weights alone. The second stage of the analysis uses the linker map files of the STM32CubeIDE projects to determine the actual binary size in flash, which also includes code, constant tables, and run-time support libraries. This explains why the measured flash

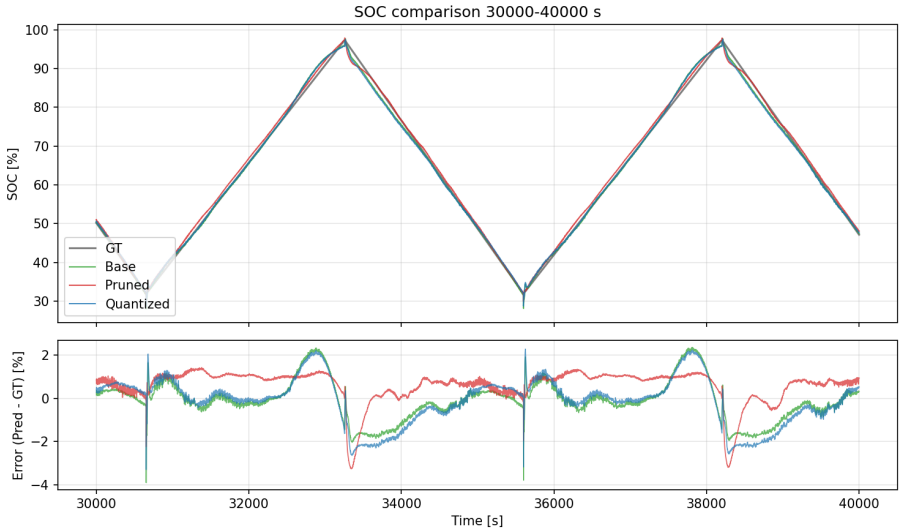

Figure 12: Detailed transient analysis of SOC streaming replay (pulse segment). Response to dynamic load pulses (30k–40k s).

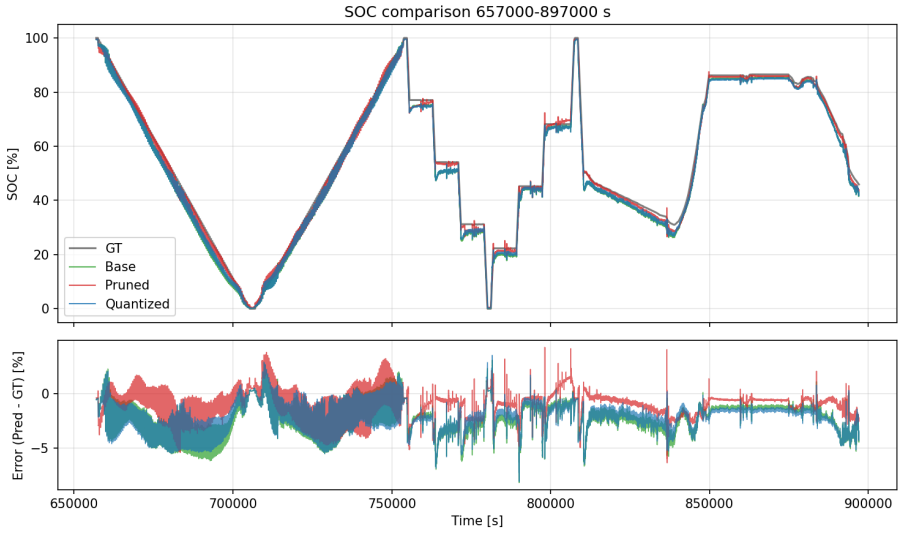

Figure 13: Detailed transient analysis of SOC streaming replay (check-up segment). Behaviour during a capacity check-up (657k–897k s).

usage in Figure 14 is systematically higher than the idealised "weights only" estimate, and why the gap is larger for the more complex SOH firmware that contains additional diagnostic and filtering logic. The RAM usage shown in the lower right panel of Figure 14 is obtained from on-device measurements of the SOH and SOC firmware and applied analogously to all STM32 deployments. At boot time the firmware reports the size of the statically allocated data and BSS sections, which correspond to persistent variables and network parameters stored in SRAM. During each benchmark inference the minimum value of the stack pointer is tracked, so that the maximum additional stack consumption of the call chain can be recovered. The total RAM requirement reported in the figure is the sum of static data and worst-case stack usage, i.e. an upper bound on the memory needed to execute one inference including the LSTM state updates and UART handling. This procedure captures the fact that pruning reduces both the number of parameters and the recurrent state dimension, whereas the weight-only quantized implementation primarily reduces recurrent-matrix storage. Activations, hidden and cell states, biases, and the MLP remain FP32. Figure 14 shows that Quantized produces the smallest flash footprint, while Pruned reduces both the parameter count and recurrent-state dimension. The measured RAM and execution-time results must nevertheless be considered separately because retaining FP32 activations and changing the kernel arithmetic prevents storage reduction from translating automatically into lower latency.

Figure 15 reports the corresponding host-side latency distributions. The latency distributions in Figure 15 reveal a distinct hierarchy. SOC inference requires approximately 0.8 ms to 7 ms, whereas the SOH variants require 12 ms to 30 ms per update. The weight-quantized SOC deployment is slowest because the hand-written mixed-precision kernel applies FP32 row scales during the recurrent matrix products. When UART framing and host processing are included, end-to-end latencies remain below roughly 20 ms for SOC and 50 ms for SOH.

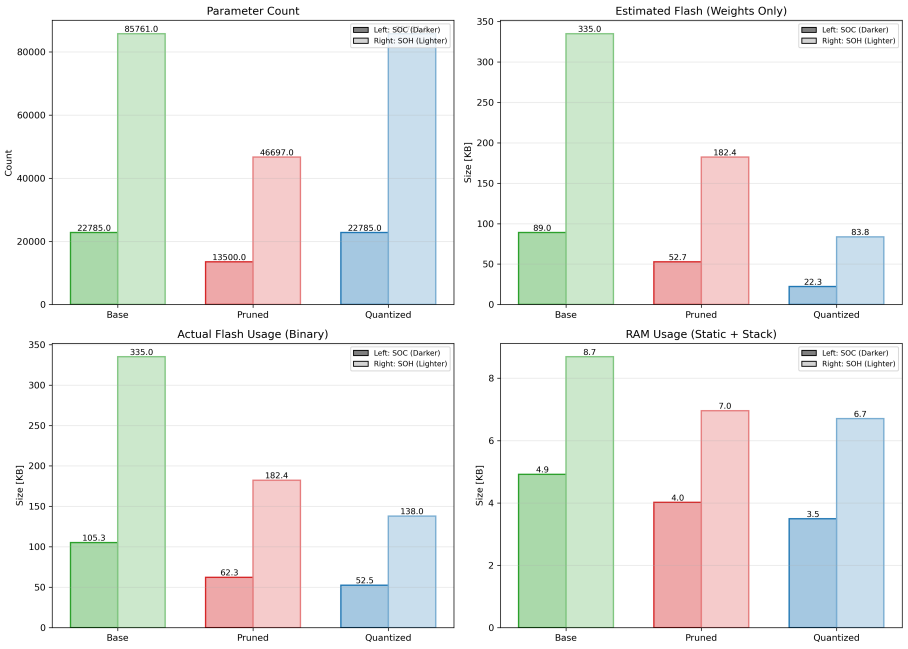

Figure 14: Resource landscape for SOC and SOH models. Comparison of parameter count, estimated Flash, actual Flash, and RAM usage. Dark bars denote SOC, light bars denote SOH.

8.6. Composite trade-off summary

To make the multi-objective trade-offs easier to interpret at a glance, we report a composite utility score U that aggregates accuracy and embedded resource indicators into a single, dimensionless index. For task-specific non-negative weights $\mathbf{w} = (w_A, w_F, w_R, w_E)$ with $\sum_j w_j = 1$, model variant v is scored as

$$U_v(\mathbf{w}) = w_A \frac{\mathrm{MAE}_v}{\mathrm{MAE}_{\mathrm{Base}}} + w_F \frac{F_v}{F_{\mathrm{Base}}} + w_R \frac{R_v}{R_{\mathrm{Base}}} + w_E \frac{E_{\mathrm{est},v}}{E_{\mathrm{est,Base}}}, \qquad (27)$$

where MAE is the streaming mean absolute error, F is measured flash, R is measured total RAM (static plus peak stack), and E_{est} is the timing-derived energy proxy. The main comparison uses equal weights, $w_A = w_F = w_R = w_E = 0.25$. In this case, U is the arithmetic mean of the four normalized objectives. By construction $U_{\mathrm{Base}} = 1$ for every valid weight vector. Lower

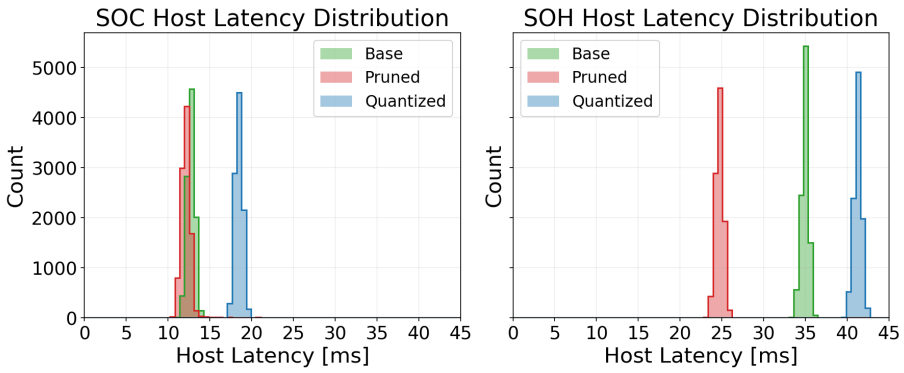

Figure 15: Latency distributions for SOC and SOH models on the STM32H7. Histograms are computed from repeated on-device measurements.

values are preferred. Because E_{est} is proportional to inference time under the fixed-power assumption, the energy term represents timing priority rather than an independent power measurement.

Sensitivity is evaluated over all 1,771 non-negative combinations on a 0.05 grid that sum to one. Appendix A.4 also varies one highlighted objective from 25 to 85 %, sharing the remaining weight equally among the other three. Pruned is top-ranked for 94.64 % of SOC combinations and 53.36 % of SOH combinations. Quantized wins 5.36 % and 17.79 %, respectively, while Base wins 0 % for SOC and 28.85 % for SOH. The ranking is therefore robust for SOC Pruned but application-dependent for SOH: Base becomes preferable when accuracy dominates, whereas Quantized becomes preferable when flash receives high weight. Table 3 summarises the resulting changes for pruning and quantization. Values are reported as SOC/SOH.

8.7. Limited software fault sensitivity

In the missing-output-report analysis, holding the preceding output for up to 10 % randomly unavailable reports changes aggregate MAE by less than 0.001 percentage points. Ten reporting gaps lasting 3600 s each increase SOC MAE by approximately 0.05 percentage points and have negligible influence on the slowly varying SOH output.

Table 3: At-a-glance effect of pruning and quantization relative to the corresponding Base model. Values are reported as SOC/SOH. ΔMAE is reported in percentage points (pp) of full scale, where a negative value indicates improved accuracy. Flash, RAM, and estimated-energy changes are relative deltas versus Base. The composite score U uses equal weights in (27) (lower is better).

Variant	ΔMAE [pp]	ΔFlash [%]	ΔRAM [%]	ΔE_{est} [%]	U
Pruned	-0.35/+0.61	-41/-46	-18/-20	-43/-44	0.71/0.91
Quantized	+0.10/+0.56	-50/-59	-20/-23	+399/+29	1.83/1.03

Figure A.20 shows the recurrent response to the one-sample input-buffer faults. Across the SOC variants, median peak deviation ranges from 3.29 to 3.82 percentage points and P95 ranges from 16.18 to 23.81 percentage points. All 30 events per SOC model meet the recovery criterion within 60 s, with median recovery times of 10.0–12.5 s. Quantized follows Base closely, whereas Pruned has a lower median peak but the largest P95 peak. For SOH, median peak deviation ranges from 0.83 to 1.59 percentage points and P95 from 3.37 to 5.85 percentage points. The share recovered within 60 s is 93.3 % for Base, 90.0 % for Pruned, and 86.7 % for Quantized. Among the recovered SOH events, median recovery is 9, 7, and 5 s, respectively.

The target-based measure in Eq. 23 complements these disturbed-clean deviations. For SOC, median ΔMAE_{61} ranges from 0.046 to 0.079 percentage points and P95 from 1.42 to 1.59 percentage points. For SOH, median changes lie between -0.0009 and 0.0012 percentage points and P95 between 0.053 and 0.122 percentage points. The small negative SOH values occur when a disturbance moves an already imperfect prediction closer to the target and are not evidence of improved fault robustness. Table A.4 gives the model-specific values. The test therefore identifies different transient response tails across the compressed variants without indicating persistent SOC divergence or non-finite outputs. Its restricted fault location and event count do not support a hardware-safety claim.

9. Discussion

The experiments show that pruning and quantization affect the SOC and SOH estimators differently. For SOC, the evaluated Pruned model obtains a 0.35-percentage-point lower MAE than Base, while Quantized remains 0.11 percentage points above Base. Together with the L2 saliency and weight-distribution diagnostics, this identifies structured pruning as the more balanced transformation for the evaluated model and split. The experimental design uses one training run and a predefined test split for each task because the focus is the embedded comparison of compression routes. Repeated-seed validation, cross-validation, and matched unpruned fine-tuning can extend the statistical assessment of the observed advantage. The result is therefore reported without assigning it to a specific regularisation mechanism.

For SOH the picture is more nuanced. Targets evolve slowly and are anchored by periodic capacity check-ups, while the displayed online estimates undergo two strong filtering stages. The filter-stage comparison shows that post-processing can alter not only absolute MAE but also the ranking between Base, Pruned, and Quantized. Compression quality should therefore be assessed on both raw and filtered outputs, and filter selection should consider response delay rather than MAE alone.

From an engineering perspective the three deployment variants represent distinct operating points. Base FP32 offers the simplest tool flow but requires the largest flash footprint. Pruning reduces the recurrent state dimension and the corresponding MLP input width, shrinking SOC flash from 105.32 to 62.27 kB and inference time from 1.40 to 0.80 ms. For SOH the corresponding changes are 335.00 to 182.41 kB and 22.73 to 12.72 ms. Within this unchanged FP32 kernel family, static MAC ratios and measured time ratios agree closely. SOC Pruned retains 54.9 % of Base MACs and 57.2 % of its time, while SOH retains 54.3 % and 56.0 %, respectively.

Weight-only quantization gives the smallest flash footprints, 52.48 kB for SOC and 138.00 kB for SOH, but changes the operation types without providing

a fully integer execution path. Base and Quantized retain the same 22,080 SOC and 85,120 SOH model MACs. In the current C expressions, each recurrent INT8 weight is cast to FP32 and multiplied by its row scale inside the innermost accumulation, adding 17,920 source-level FP32 scale multiplications for SOC and 68,608 for SOH. Treating one MAC and one additional multiplication as one operation each gives a static Quantized/Base ratio of approximately 1.81 for both tasks, whereas the measured time ratios are 4.99 for SOC and 1.29 for SOH.

The discrepancy is expected because the count omits conversion, indexing, loop, load/store, nonlinear-function, and cache/Flash effects and because the `-O0` kernel uses no optimised integer dot-product path. For the larger SOH model, reduced recurrent-weight traffic may offset more of this arithmetic overhead than for SOC. Memory bandwidth was not measured, so this remains an implementation-based interpretation rather than a causal attribution. Appendix A.9 places the static counts beside the measured DWT times.

The present kernel prioritises a transparent implementation of the mixed-precision path. Potential improvements include accumulating each row before applying its shared scale, fusing scale and bias work, using DSP-optimised or integer dot-product kernels, and quantizing activations and recurrent states to enable a complete integer path. Operation-level cycle attribution and memory-bandwidth profiling are reserved for future implementation studies. The present results therefore establish total kernel timing for the tested firmware but not the fraction caused by conversion, arithmetic, or memory access.

The distinction between system latency, inference time, and estimated energy is particularly relevant for resource-constrained BMS. The UART-driven tests report end-to-end latency, while on-device inference spans 0.80–6.99 ms for SOC and 12.72–29.21 ms for SOH. The reported E_{est} values are obtained by multiplying these times by an assumed constant 0.5 W. They do not isolate CPU, flash, SRAM, UART, or regulator contributions. Consequently, the results establish timing and memory trade-offs on the STM32H753ZI, but not a component-level energy budget or a conclusion that flash access dominates

consumption.

The utility score makes the competing effects of compression directly comparable by expressing accuracy, flash, RAM, and the timing-derived energy proxy relative to the same Base model. A value below one indicates that the combined advantages outweigh the disadvantages for the selected priorities, while a value above one indicates an overall deterioration. With equal weights, $U = 0.71$ identifies SOC Pruned as a balanced improvement, whereas $U = 1.83$ shows that the flash reduction of SOC Quantized does not compensate for its timing overhead. The SOH scores remain close to one, indicating a genuine trade-off without one consistently dominant variant. The weight-grid analysis adds the important information of how stable these conclusions are when application priorities change. SOC Pruned remains preferred across most weight combinations, while the preferred SOH model depends more strongly on whether accuracy or flash receives greater emphasis. The utility analysis therefore supports a transparent selection of the model variant and distinguishes robust recommendations from priority-dependent decisions.

The implementation-level storage reduction follows directly from the architecture. Removing the same number of hidden channels or storing the same recurrent matrices as INT8 therefore produces predictable parameter-count changes for a fixed model topology. The preservation of estimation accuracy depends more strongly on the cell chemistry, operating conditions, reference construction, and training data. This study quantifies the trade-off for JGC LFP cells at 25 °C. Existing embedded studies across different cell chemistries demonstrate that compact data-driven estimators are broadly feasible [11, 24, 49]. Validation with additional cell chemistries is needed to determine how closely their attainable MAE and preferred compression level follow the trends observed here.

The present study examines two post-training compression routes. The first is one-shot structured pruning with short fine-tuning, and the second is recurrent-weight quantization of the original FP32 model. Quantization-aware training and pruning-aware or iterative training can in principle recover accu-

racy by adapting parameters to the imposed representation [47, 50]. Evaluating these strategies requires a separate training study and is reserved for future work. The current comparison therefore addresses deployable post-training transformations, not the best achievable result of jointly trained compression.

The analytical MAC relation provides a platform-independent estimate of how structured pruning changes model complexity. The measured flash and timing ratios complement this estimate for the evaluated STM32 implementation and build configuration. Their transfer to other embedded platforms depends on clock rate, floating-point and signal-processing support, compiler optimisation, cache, and memory organisation. Benchmarks on additional controller architectures can therefore quantify how these implementation characteristics translate the same model reduction into practical resource savings.

Preparing the derivative features on the host separates feature construction from neural-network inference and allows the measured timing differences to be attributed directly to pruning and quantization. The long-horizon analysis evaluates continuous inference trajectories, while the transient input-buffer analysis perturbs a measurement before inference and follows its effect through the recurrent state. The latter distinguishes immediate peak deviation, recovery, and target-MAE change. It indicates similar SOC behaviour for Base and Quantized, a larger upper-tail SOC response for Pruned, and a larger share of SOH events without confirmed 60-s recovery for Quantized. A complete embedded front-end and safety assessment can build on these results by additionally covering variable sampling, derivative filtering, recurrent-state and weight memory, activations, communication, and MCU memory.

10. Conclusion and Outlook

This study set out to quantify how two transparent compression routes change the accuracy and embedded cost of stateful SOC and SOH estimators under a common STM32 benchmark. Its contribution is an auditable comparison from trained LSTM + MLP models to continuous C inference with consistent ac-

curacy, memory, and timing indicators. Estimated energy per inference remains a constant-power timing proxy and is not an independent power measurement.

For SOC, Pruned reduces flash by 40.9 %, RAM by approximately 18 %, and inference time by 42.9 %, while its MAE is 0.35 percentage points lower than Base for the evaluated model and split. Quantized reduces flash by 50.2 % and RAM by approximately 20 %, but increases inference time by 399 % and MAE by 0.11 percentage points in the mixed-precision software kernel. For SOH, Pruned reduces flash by 45.5 %, RAM by approximately 20 %, and inference time by 44.0 %, with a 0.61-percentage-point MAE increase. Quantized reduces flash by 58.8 % and RAM by approximately 23 %, while increasing inference time by 28.5 % and MAE by 0.56 percentage points. Under the fixed-power assumption, the energy proxy changes by the same percentages as inference time.

These results identify structured pruning as the more balanced operating point for the tested firmware, whereas recurrent-weight quantization is attractive primarily when flash capacity dominates and its dequantization overhead is acceptable. Ten consecutive replay segments show no quantization-specific monotonic deviation from Base. After transient input-buffer bit flips, every SOC variant recovers in all 30 events within 60 s, while 6.7–13.3 % of the SOH events do not meet the defined recovery criterion within this horizon. The SOH filter analysis further shows that compression must be assessed together with post-processing delay. The evaluated LFP trajectories, model configurations, STM32H753ZI implementation, and compiler settings establish the trade-off under one complete deployment scenario. Validation across additional cell chemistries, embedded platforms, model initialisations, and internal fault classes can determine how broadly the observed trends transfer.

The first priority for future work is to optimise and profile the mixed-precision kernel through row-scale hoisting, operator fusion, DSP-oriented dot products, and activation/state quantization, while measuring per-model board power with external instrumentation. A second priority is external validation across random seeds, temperatures, cell chemistries, and embedded platforms,

including direct comparison with quantization-aware and pruning-aware training. Finally, internal weight, activation, recurrent-state, sensor, communication, and MCU fault injection is required before safety-related claims can be made. Joint pruning and quantization remains a promising route for combining a reduced state dimension with compact recurrent storage.

Appendix A. Supplementary compression and robustness analyses

This appendix presents supporting analyses of pruning structure, temporal stability, filter interaction, utility sensitivity, transient input faults, model-complexity scaling, mixed precision, and kernel operation counts.

Appendix A.1. Structured-pruning diagnostics

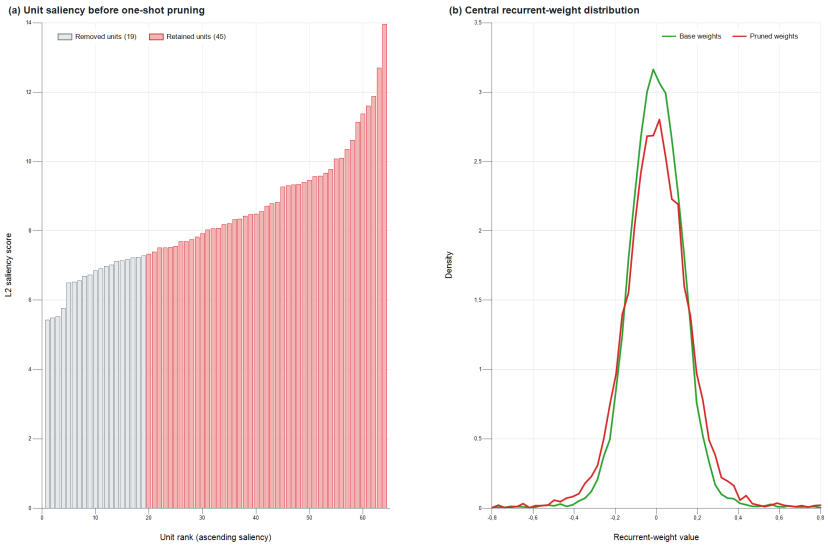

Figure A.16: Diagnostics for the one-shot SOC pruning operation. (a) L2 saliency from Eq. 16, sorted in ascending order. The 19 lowest-ranked units are removed and 45 units are retained. (b) Density of the central recurrent-weight range for the Base and Pruned models. The analysis confirms application of the specified ranking criterion and makes the resulting weight selection transparent. Repeated training runs can extend this structural evidence toward a statistical assessment of the observed MAE difference.

Appendix A.2. Long-horizon temporal stability

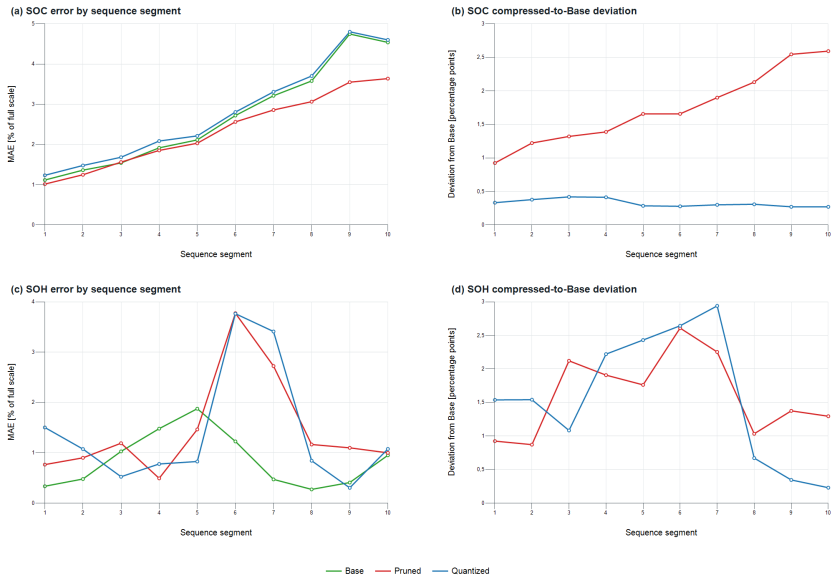

Figure A.17: Temporal error and model-to-Base deviation over ten consecutive, equal-count segments of the full-stream replay. Recurrent states are not reset at segment boundaries. Panels (a) and (c) show target MAE for SOC and SOH. Panels (b) and (d) show the mean absolute deviation of Pruned and Quantized from Base. The absence of monotonic late-sequence divergence demonstrates stable relative behaviour across the complete evaluated replay. Further sequences can extend the tested operating horizon.

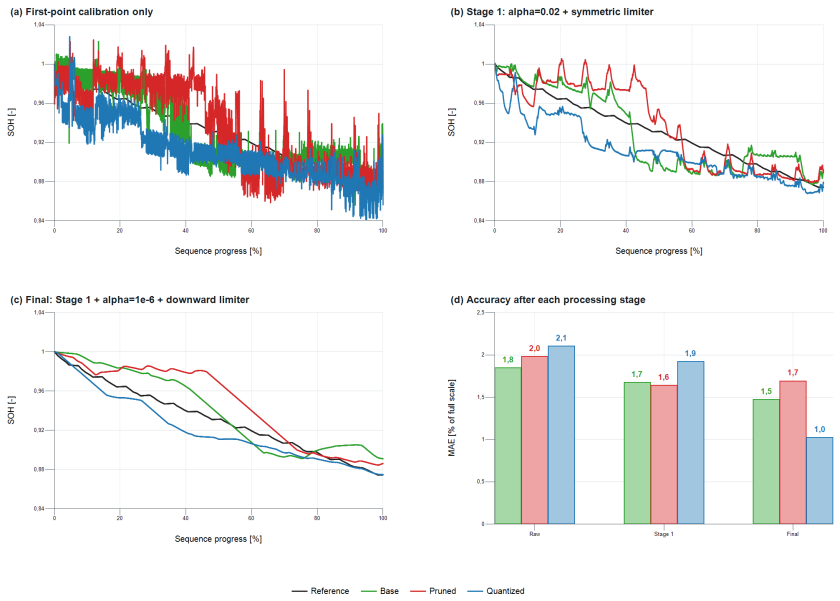

Figure A.18: Comparison of SOH compression and filtering for the Base, Pruned, and Quantized C models on a common input sequence. (a) Outputs after first-point alignment but before temporal filtering. (b) Stage 1 with $\alpha_1 = 0.02$ and the symmetric limiter. (c) Final sequential output after applying stage 2 with $\alpha_2 = 10^{-6}$ and the downward limiter to stage 1. (d) MAE after each processing stage. At 1 Hz, the EMA-only 90 % response times are approximately 114 s and 26.65 days, respectively. The nonlinear limiters have no single cutoff frequency. All three variants use the same input and processing order to isolate the interaction between compression and filtering.

Appendix A.4. Utility-weight sensitivity

Figure A.19: Sensitivity of the utility ranking to application priorities. Panels (a) and (b) vary the highlighted objective from 25 to 85 % and divide the remaining weight equally among the other three objectives. Each square identifies the lowest-utility model. At 25 %, all four objectives have equal weight. Panel (c) gives the winning share across all 1,771 non-negative 0.05-spaced weight combinations that sum to one. The energy objective uses the timing-derived E_{est} proxy and is therefore not an independent power measurement.

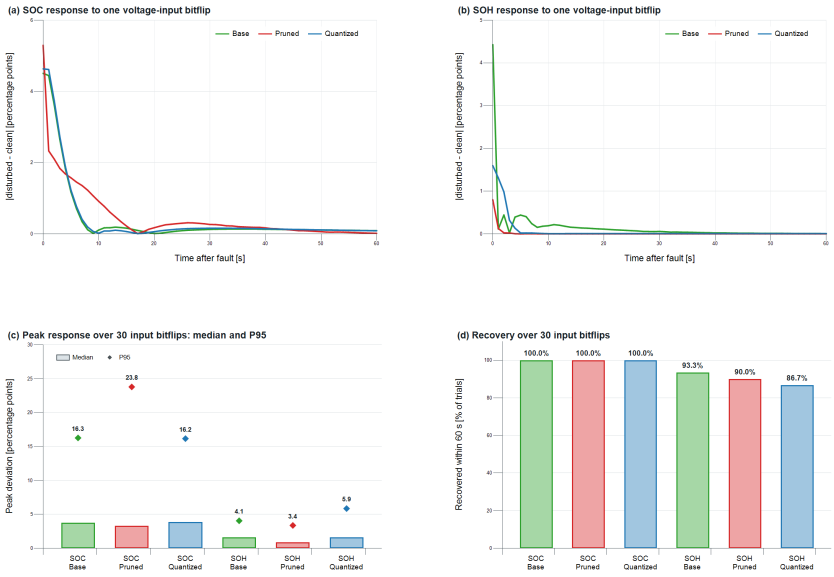

Figure A.20: Stateful response to transient software input-buffer corruption. Thirty independent events per task and model flip bit 22, the highest-order FP32 fraction bit, in voltage, current, or temperature for one 1-Hz sample, with ten events per input signal. The clean and disturbed branches start from the same recurrent state and then receive the same 60 unmodified follow-up inputs, corresponding to 60 s, without a state reset. (a) and (b) show the absolute disturbed-clean deviation for one representative voltage event. (c) summarises the median peak and P95 peak across all 30 events. (d) gives the fraction that meets the sustained recovery criterion within 60 s. The figure characterises propagation through the recurrent state, while Table A.4 reports the corresponding change in target MAE.

Table A.4: Change in target MAE over the window comprising one corrupted sample and 60 clean follow-up samples. Values are percentage points and summarise 30 independent input-bitflip events per task and model. The change is defined by Eq. 23. Negative values indicate that the disturbed output happens to lie closer to the target in part of the window.

Task	Model	Median $\Delta\mathrm{MAE}_{61}$ [pp]	P95 $\Delta\mathrm{MAE}_{61}$ [pp]
SOC	Base	0.0740	1.4235
SOC	Pruned	0.0464	1.5867
SOC	Quantized	0.0785	1.4314
SOH	Base	−0.0009	0.1224
SOH	Pruned	−0.00004	0.0533
SOH	Quantized	0.0012	0.1055

Appendix A.6. Analytical model-complexity scaling

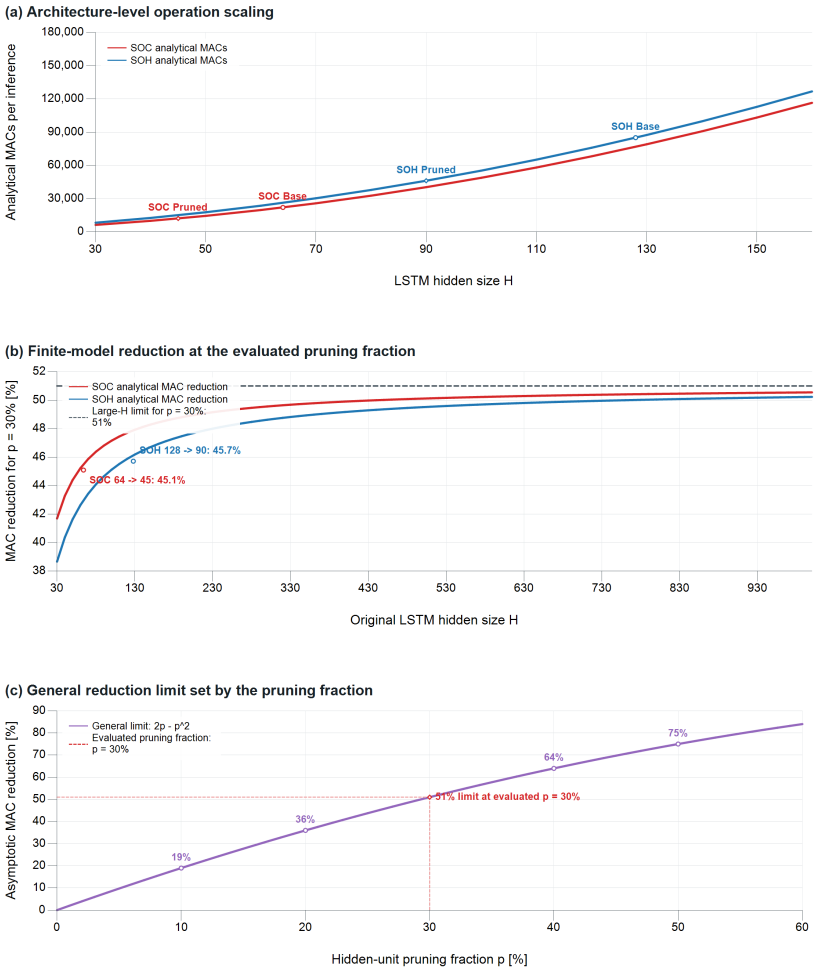

Figure A.21: Architecture-level operation scaling from Eq. 14. (a) Analytical MACs per streaming inference over LSTM hidden size, with markers for the implemented SOC and SOH Base and Pruned dimensions. (b) Finite-model reduction for the evaluated pruning fraction $p = 0.30$, including the implemented SOC and SOH points and the 51 % large-H limit. (c) General relation between the hidden-unit pruning fraction and the asymptotic reduction $2p - p^2$. The highlighted 30 % operating point has a theoretical limit of 51 %, while the corresponding limits for 10, 20, 40, and 50 % pruning are 19, 36, 64, and 75 %. Linear input and MLP terms together with the fixed output term keep finite models below the respective limit.

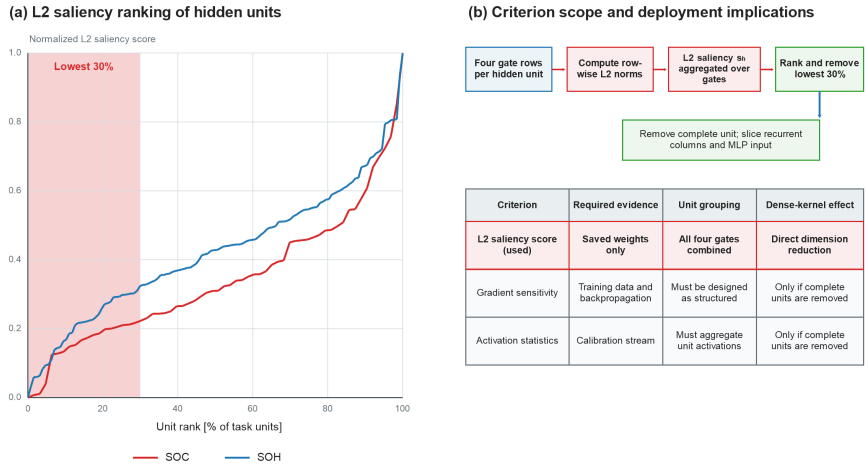

Figure A.22: Scope of the structured pruning criterion. (a) Normalised L2 saliency ranks for the SOC and SOH Base hidden units. The lowest 30 % define the task-specific removal sets. (b) The implemented path aggregates row-wise L2 norms across all four gates and removes complete recurrent units together with associated recurrent columns and MLP inputs. The comparison makes the reproducibility and dense-kernel benefit of the L2 criterion explicit and provides a reference for future comparisons with gradient- and activation-based rankings.

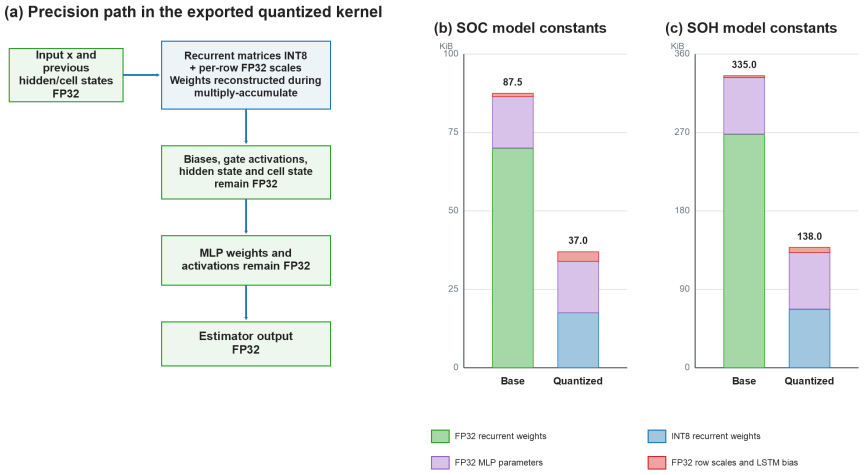

Figure A.23: Verified precision path and raw model-constant composition. (a) Only recurrent input-to-hidden and hidden-to-hidden matrices are stored as INT8. Per-row scales, biases, gate activations, hidden/cell states, the MLP, and the estimator output remain FP32. (b,c) Analytical model constants for SOC and SOH Base and Quantized variants. The comparison shows how concentrating quantization on the dominant recurrent matrices reduces persistent model storage and where activation and state quantization could provide additional runtime savings.

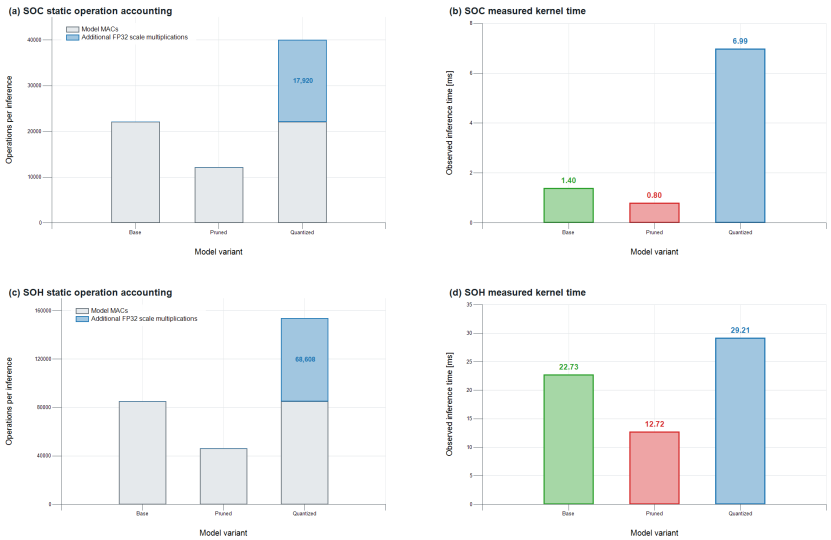

Figure A.24: Static source-level operation accounting beside measured STM32H753ZI kernel time. Panels (a,c) show model MACs and the additional FP32 row-scale multiplications implied by the current per-weight C expressions. Panels (b,d) show DWT measurements averaged over 10,000 streaming inferences. Their comparison reveals where analytical operation counts explain the timing trend and where conversion, indexing, loop, load/store, nonlinear-function, compiler, and memory-system effects create additional optimisation opportunities.

Funding

This research was funded by the German Federal Ministry for Education and Research (BMBF), grant number 03SF0684A (MG FARM). The authors are responsible for the content. The work was further supported by the German Research Foundation and the Open Access Publication Fund of the Technical University of Berlin.

Acknowledgements

We acknowledge the High Performance Computing Service of the Technical University of Berlin for providing the computational resources used in the pruning and quantization experiments. The authors thank Nils Strassenburg (Hasso-Plattner-Institut, Potsdam) for helpful discussions on pruning and quantization, Mano Schmitz for the collaboration on the dataset creation, and Max Mönikes and Pavel Aleinikau (Technical University of Berlin) for their collaboration on the BMS hardware.

CRediT authorship contribution statement

Florian Rzepka: Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing–original draft preparation, writing–review and editing, visualization.

Julia Kowal: Resources, supervision, project administration, funding acquisition, writing–review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used ChatGPT, DeepL, and Grammarly in order to improve the linguistic quality and style of the manuscript. After using these tools, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Data availability

The LFP DoE Cycle Ageing Dataset used in this work can be obtained from the DOI repository in [15]. The landing page provides licensing information and download instructions.

References

[1] J. Yao, J. Kowal, Towards a smarter battery management system: A critical review on deep learning-based state of charge estimation of lithium-ion batteries, Energy and AI 21 (2025) 100585. doi:10.1016/j.egyai.2025.100585.

[2] A. M. S. M. H. S. Attanayaka, J. P. Karunadasa, K. T. M. U. Hemapala, Department of Electrical Engineering, University of Moratuwa, Moratuwa, Sri Lanka, Estimation of state of charge for lithium-ion batteries - A Review, AIMS Energy 7 (2) (2019) 186–210. doi:10.3934/energy.2019.2.186.

[3] M. Berecibar, I. Gandiaga, I. Villarreal, N. Omar, J. Van Mierlo, P. Van Den Bossche, Critical review of state of health estimation methods of Li-ion batteries for real applications, Renewable and Sustainable Energy Reviews 56 (2016) 572–587. doi:10.1016/j.rser.2015.11.042.

[4] P. Shrivastava, P. A. Naidu, S. Sharma, B. K. Panigrahi, A. Garg, Review on technological advancement of lithium-ion battery states estimation methods for electric vehicle applications, Journal of Energy Storage 64 (2023) 107159. doi:10.1016/j.est.2023.107159.

[5] F. Guo, L. D. Couto, G. Mulder, K. Trad, G. Hu, O. Capron, K. Haghverdi, A systematic review of electrochemical model-based lithium-ion battery state estimation in battery management systems, Journal of Energy Storage 101 (2024) 113850. doi:10.1016/j.est.2024.113850.

[6] LEAP-RE, MG FARM Smart stand-alone micro-grids as a solution for agriculture farms electrification, https://www.leap-re.eu/mg-farm/.

[7] Microenergy, MG-FARM Powering Sustainable Agriculture with Smart Micro-Grids, https://mg-farm.microenergy-consulting.com/.

[8] A. Aqachtoul, K. Karam, A. Elamrani, Y. Alidrissi, M. Najib, A. Rochd, M. Bakhouya, MGFARM: An IoT and Edge-Driven Framework for Smart and Sustainable Agricultural Practices, in: Y. Mejdoub, A. Elamri, M. Kardouchi (Eds.), Connected Objects, Artificial Intelligence, Telecommunications and Electronics Engineering, Vol. 1584, Springer Nature Switzerland, Cham, 2026, pp. 419–425. doi:10.1007/978-3-032-01536-5_64.

[9] Technische Universität Berlin, MG Farm Smart micro-grids as a solution for agriculture farms electrification, https://www.tu.berlin/eet/forschung/projekte/mg-farm.

[10] G. Crocioni, D. Pau, J.-M. Delorme, G. Gruosso, Li-Ion Batteries Parameter Estimation With Tiny Neural Networks Embedded on Intelligent IoT Microcontrollers, IEEE Access 8 (2020) 122135–122146. doi:10.1109/ACCESS.2020.3007046.

[11] S. Giazitzis, M. Sakwa, S. Leva, E. Ogliari, S. Badha, F. Rosetti, A Case Study of a Tiny Machine Learning Application for Battery State-

of-Charge Estimation, Electronics 13 (10) (2024) 1964. doi:10.3390/electronics13101964.

[12] D. P. Pau, A. Aniballi, Tiny Machine Learning Battery State-of-Charge Estimation Hardware Accelerated, Applied Sciences 14 (14) (2024) 6240. doi:10.3390/app14146240.

[13] M. A. Kamali, A. C. Caliwag, W. Lim, Novel SOH Estimation of Lithium-Ion Batteries for Real-Time Embedded Applications, IEEE Embedded Systems Letters 13 (4) (2021) 206–209. doi:10.1109/LES.2021.3078443.

[14] M. Naguib, P. Kollmeyer, C. Vidal, J. Duque, O. Gross, A. Emadi, Microprocessor Execution Time and Memory Use for Battery State of Charge Estimation Algorithms, in: WCX SAE World Congress Experience, Detroit & Online, Michigan, United States, 2022, pp. 2022–01–0697. doi:10.4271/2022-01-0697.

[15] F. Rzepka, Design of Experiments for Cyclic Aging of LFP 18650 Cells: Impact of Charge/Discharge C-Rates, Depth of Discharge, and Temperature on Battery Degradation (Nov. 2025). doi:10.14279/DEPOSITONCE-24800.

[16] S. Wang, C. Wang, P. Takyi-Aninakwa, S. Jin, C. Fernandez, Q. Huang, An improved parameter identification and radial basis correction-differential support vector machine strategies for state-of-charge estimation of urban-transportation-electric-vehicle lithium-ion batteries, Journal of Energy Storage 80 (2024) 110222. doi:10.1016/j.est.2023.110222.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352152X23036216

[17] S. Wang, F. Wu, P. Takyi-Aninakwa, C. Fernandez, D.-I. Stroe, Q. Huang, Improved singular filtering-Gaussian process regression-long short-term memory model for whole-life-cycle remaining capacity estimation of lithium-ion batteries adaptive to fast aging and multi-current variations, Energy 284 (2023) 128677. doi:10.1016/j.energy.2023.128677.

URL https://linkinghub.elsevier.com/retrieve/pii/S0360544223020716

[18] Y. Gao, K. Liu, C. Zhu, X. Zhang, D. Zhang, Co-Estimation of State-of-Charge and State-of- Health for Lithium-Ion Batteries Using an Enhanced Electrochemical Model, IEEE Transactions on Industrial Electronics 69 (3) (2022) 2684–2696. doi:10.1109/TIE.2021.3066946.

[19] Y. Gao, G. L. Plett, G. Fan, X. Zhang, Enhanced state-of-charge estimation of LiFePO4 batteries using an augmented physics-based model, Journal of Power Sources 544 (2022) 231889. doi:10.1016/j.jpowsour.2022.231889.

[20] J. Shen, W. Ma, J. Xiong, X. Shu, Y. Zhang, Z. Chen, Y. Liu, Alternative combined co-estimation of state of charge and capacity for lithium-ion batteries in wide temperature scope, Energy 244 (2022) 123236. doi:10.1016/j.energy.2022.123236.

[21] C. Wang, N. Cui, Z. Cui, H. Yuan, C. Zhang, Fusion estimation of lithium-ion battery state of charge and state of health considering the effect of temperature, Journal of Energy Storage 53 (2022) 105075. doi:10.1016/j.est.2022.105075.

[22] W. Sun, B. Zhou, H. Xu, W. Yao, Y. Guo, Z. Yang, State of Charge Estimation of Sodium-Ion Batteries Based on N-BEATS Neural Network, in: 2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT), IEEE, Jiaxing, China, 2023, pp. 1395–1401. doi:10.1109/ACAIT60137.2023.10528411.

[23] W. Sun, H. Xu, B. Zhou, Y. Guo, Y. Tang, W. Yao, Z. Yang, State-of-charge estimation of sodium-ion batteries: A fusion deep learning approach, Journal of Energy Storage 91 (2024) 111527. doi:10.1016/j.est.2024.111527.

[24] W. Li, N. Sengupta, P. Dechent, D. Howey, A. Annaswamy, D. U. Sauer, Online capacity estimation of lithium-ion batteries with deep long short-term memory networks, Journal of Power Sources 482 (2021) 228863. doi:10.1016/j.jpowsour.2020.228863.

[25] ONNX Runtime: High-performance inference engine, https://onnxruntime.ai/.

[26] NVIDIA TensorRT: High-Performance Deep Learning Inference, https://developer.nvidia.com/tensorrt.

[27] X-CUBE-AI: STM32Cube Expansion Package for Artificial Intelligence, https://www.st.com/en/embedded-software/x-cube-ai.html.

[28] J. Schulze, N. Strassenburg, T. Rabl, PQ Bench: Benchmarking Pruning and Quantization Techniques, in: Proceedings of the Workshop on Data Management for End-to-End Machine Learning, ACM, Berlin Germany, 2025, pp. 1–6. doi:10.1145/3735654.3735940.

[29] JGNE, JGCFR18650-1800mAh-3.2V Product Specification.

[30] D. Kucevic, B. Tepe, S. Englberger, A. Parlikar, M. Mühlbauer, O. Bohlen, A. Jossen, H. Hesse, Standard battery energy storage system profiles: Analysis of various applications for stationary energy storage systems using a holistic simulation framework, Journal of Energy Storage 28 (2020) 101077. doi:10.1016/j.est.2019.101077.

[31] K. Siebertz, D. Van Bebber, T. Hochkirchen, Statistische Versuchsplanung, Springer Berlin Heidelberg, Berlin, Heidelberg, 2017. doi:10.1007/978-3-662-55743-3.

[32] P. J. Huber, E. M. Ronchetti, Robust Statistics, 1st Edition, Wiley Series in Probability and Statistics, Wiley, 2009. doi:10.1002/9780470434697.

[33] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, A. Müller, J. Nothman, G. Louppe, P. Prettenhofer, R. Weiss,

V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, Scikit-learn: Machine Learning in Python (2012). doi:10.48550/ARXIV.1201.0490.

[34] G. L. Plett, Battery Management Systems: Volume 1: Battery Modeling, Artech House Power Engineering and Power Electronics, Artech House, Boston London, 2015.

[35] Z. C. Lipton, J. Berkowitz, C. Elkan, A Critical Review of Recurrent Neural Networks for Sequence Learning (2015). doi:10.48550/ARXIV.1506.00019.

[36] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: An Imperative Style, High-Performance Deep Learning Library (Dec. 2019). arXiv:1912.01703, doi:10.48550/arXiv.1912.01703.

[37] I. Loshchilov, F. Hutter, Decoupled Weight Decay Regularization (2017). doi:10.48550/ARXIV.1711.05101.

[38] I. Loshchilov, F. Hutter, SGDR: Stochastic Gradient Descent with Warm Restarts (2016). doi:10.48550/ARXIV.1608.03983.

[39] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, H. Wu, Mixed Precision Training (2017). doi:10.48550/ARXIV.1710.03740.

[40] R. Pascanu, T. Mikolov, Y. Bengio, On the difficulty of training Recurrent Neural Networks (2012). doi:10.48550/ARXIV.1211.5063.

[41] S. Han, J. Pool, J. Tran, W. J. Dally, Learning both Weights and Connections for Efficient Neural Networks (2015). doi:10.48550/ARXIV.1506.02626.

[42] D. Blalock, J. J. G. Ortiz, J. Frankle, J. Guttag, What is the State of Neural Network Pruning? (2020). doi:10.48550/ARXIV.2003.03033.

[43] W. Wen, C. Wu, Y. Wang, Y. Chen, H. Li, Learning Structured Sparsity in Deep Neural Networks (2016). doi:10.48550/ARXIV.1608.03665.

[44] E. Lobacheva, N. Chirkova, A. Markovich, D. Vetrov, Structured Sparsification of Gated Recurrent Neural Networks, Proceedings of the AAAI Conference on Artificial Intelligence 34 (04) (2020) 4989–4996. doi:10.1609/aaai.v34i04.5938.

[45] S. Narang, E. Elsen, G. Diamos, S. Sengupta, Exploring Sparsity in Recurrent Neural Networks (2017). doi:10.48550/ARXIV.1704.05119.

[46] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference (2017). doi:10.48550/ARXIV.1712.05877.

[47] R. Krishnamoorthi, Quantizing deep convolutional networks for efficient inference: A whitepaper (2018). doi:10.48550/ARXIV.1806.08342.

[48] STM32 Nucleo-144 boards (Dec. 2025).

[49] F. Rzepka, P. Hematty, M. Schmitz, J. Kowal, Neural Network Architecture for Determining the Aging of Stationary Storage Systems in Smart Grids, Energies 16 (17) (2023) 6103. doi:10.3390/en16176103.

[50] W. Wen, Y. He, S. Rajbhandari, M. Zhang, W. Wang, F. Liu, B. Hu, Y. Chen, H. Li, Learning Intrinsic Sparse Structures within Long Short-Term Memory (2017). doi:10.48550/ARXIV.1709.05027.